

基于 CRITIC-TOPSIS 与混合整数规划的生产企业原材料订购与运输优化

摘要

随着供应链全球化发展与行业成本竞争的持续加剧，原材料采购、运输、库存管控面临**供应商履约不稳定、运输存在损耗、多类原料成本与产能贡献差异显著**等现实痛点。本文聚焦生产企业原材料订购与运输优化问题，基于企业 402 家原材料供应商、8 家转运商近五年 240 周的订货、供货及运输损耗原始数据，逐层进行探讨。模型建立在历史供货、损耗统计特征可复现的前提假设之上，构建出“综合评价—混合整数规划—瓶颈诊断”的研究模型，可为企业采购管理与物流调度的实际决策提供量化支撑。

针对问题一，通过消耗系数把三类原料折算为等效产品当量实现量纲统一，从供给规模、供给波动、供货可靠度、供货持续性、应急保障以及断供风险六个维度构建评价指标体系。采用 CRITIC 法客观赋权，兼顾指标对比强度 σ_k 和冲突性 C_k ，弥补熵权法受指标相关性干扰的缺陷，结合 TOPSIS 贴近度完成供应商排序。结果显示供货可靠度权重 0.3459 占比最大，供货持续性 0.1945、断供风险 0.1500 次之；前 50 位核心供应商包含 A 类 14 家、B 类 18 家、C 类 18 家。经熵权-TOPSIS 交叉验证，样本重合率 82%；权重 $\pm 20\%$ 扰动下重合率高于 80%，评价结果可靠。

针对问题二，以“活跃周平均供货量”刻画供应商可持续供货能力，将历史峰值口径导致的 $N=1$ 退化解与全周均值口径的不可行性去除，并建立 0-1 混合整数线性规划（MILP）：先确定最少供应商数 $N=22$ （A 类 9 家、B 类 6 家、C 类 7 家），再在该数量下最小化采购成本得到 24 周订购方案（采购成本 494 950、总原料量 441 176 m^3 ），最后以总损耗最小为目标进行转运商指派。结果表明综合损耗率仅 0.31%，远低于 8 家转运商平均损耗率 1.33%；且供应商数由 22 增至 30 时总成本仅下降 0.07%，验证了最少供应商数兼具可行性与经济性。

针对问题三，本文将单位运输仓储费用纳入模型目标函数。三类原料生产单位产品的采购成本接近，但 A 类原料消耗系数更低，单位产品对应的运储成本显著小于 C 类；运储费用构成了偏好 A 类的核心经济激励，目标函数最小化总成本会自动向 A 类倾斜。优化供应商采购结构后，A 类原料采购占比从 47.6% 提升至 49.8%。C 类原料采购占比从 25.7% 下降至 20.5%。在 24 周的生产周期内，企业整体原料采购总量减少 0.63%。原料综合运输损耗率降至 0.309%。本次优化同时达成了原料结构升级、成本压缩、损耗降低的三重优化效果。

针对问题四，本文将周产能设为可变变量，以产能最大化为核心目标求解。保持现有 22 家固定供应商合作规模时，企业实际最大产能为 28214 立方米，较原有水平提升 0.1%。若启用全部供应商资源，理论最大产能可达 31042 立方米，产能提升 10.1%。**本文采用约束松弛近似影子价格的方式识别系统瓶颈：分别将供应商总供货能力扩大 10%、转运总运力扩大 25%，观察产能输出变化幅度，判断约束松紧程度。**结果显示瓶颈为供应商供货能力，转运运力利用率仅约 50% 并未饱和；结合首周安全库存堆积约束推导出产能上限解析公式，计算结果和模型输出匹配。需要注意最大产能仅代表产出上限，并未同步评估该情景下采购成本与综合经营效益。

关键词：供应商评价；CRITIC-TOPSIS；混合整数线性规划；多目标优化；瓶颈分析

一、问题重述

某生产企业需要三类原材料 A、B、C 生产同一种产品，每周产能为 2.82 万立方米。生产每立方米产品分别消耗 0.6 立方米 A 类、0.66 立方米 B 类、0.72 立方米 C 类原材料；A 类采购单价比 C 类高 20%，B 类比 C 类高 10%。企业每年按 48 周安排生产，为保证生产连续，需保持不少于两周生产需求的原材料库存。企业共有 402 家原材料供应商和 8 家转运商。

单家转运商周运力上限为 6000 立方米、转运商存在运输损耗，原则上单家供应商的周供货量由一家转运商承运。

附件 1 给出近 5 年 402 家供应商每周的订货量与供货量的数据，附件 2 给出近 5 年 8 家转运商的运输损耗率数据。现在需要完成如下四个问题。

1. 对附件一中的 402 家供应商的供货特征进行量化分析，建立“保障企业生产重要性”的评价模型，并确定 50 家最重要的供应商。
2. 结合问题一得到的供应商重要排序确定至少选择多少家供应商才能满足生产需求，并给出未来 24 周最经济的订购方案与损耗最少的转运方案。
3. 企业为压缩生产成本，计划尽量多采购 A 类、少采购 C 类原材料以减少转运及仓储成本，同时希望转运损耗尽量少，制定新的订购与转运方案并分析实施效果。
4. 企业已具备提高产能的潜力，确定现有条件下每周产能最多能提高多少，并给出对应的方案，对方案的实施进行分析。

二、问题分析

问题一是多指标综合评价与排序问题。主要建立一个关于供应商重要性的指标体系，接着从客观上来赋权排序。重要性体现在多种维度，包括生产的规模、稳定、应急等，同时，为消除三类原料类型差异带来的影响，将各类原材料统一折算为产能贡献口径来计算，在赋权方法的选择上，熵权法只依靠指标数据离散程度确定权重，易到指标间相关性的干扰。因此本文选用 CRITIC 法进行客观赋权，该方法可以同时兼顾指标的对比强度与冲突。为保证评价结果的可靠性，另外引入熵权法、权重扰动做交叉验证。本次综合评价得到的供应商排序结果，也可作为第二问供应商筛选的候选集合。

问题二属于包含供应商筛选、订货量确定、转运指派的三层组合优化问题，求解主要存在两处难点，一是供应商的选取数量 N 为决策变量，需要保障解的可行性；二是供应商供货能力的刻画对 N 取值有影响，直接取历史峰值会造成能力高估，全周期平均又会被未订货周的数据拉低。本文以活跃周平均供货量定义可持续供货能力，先求最小供应商数量，再构建混合整数规划模型来完成订货与转运方案优化。

问题三是问题二的延伸，在压缩成本的同时兼顾“多采 A 类、少采 C 类”与降低损耗的需求。本文不设置主观惩罚参数，通过把运储费用 h 写入目标函数，依靠成本结构内生得到多 A 少 C 的采购结果；求解采用字典序，优先保证成本最优，再实现损耗最小。

问题四围绕技术改造后的产能上限展开，将周产能 K 作为连续决策变量求最大化，受供应商供货、转运运力、安全库存约束；利用约束松弛的影子价格识别系统瓶颈，区分不更换供应商的现实条件与可新增供应商的理论条件。

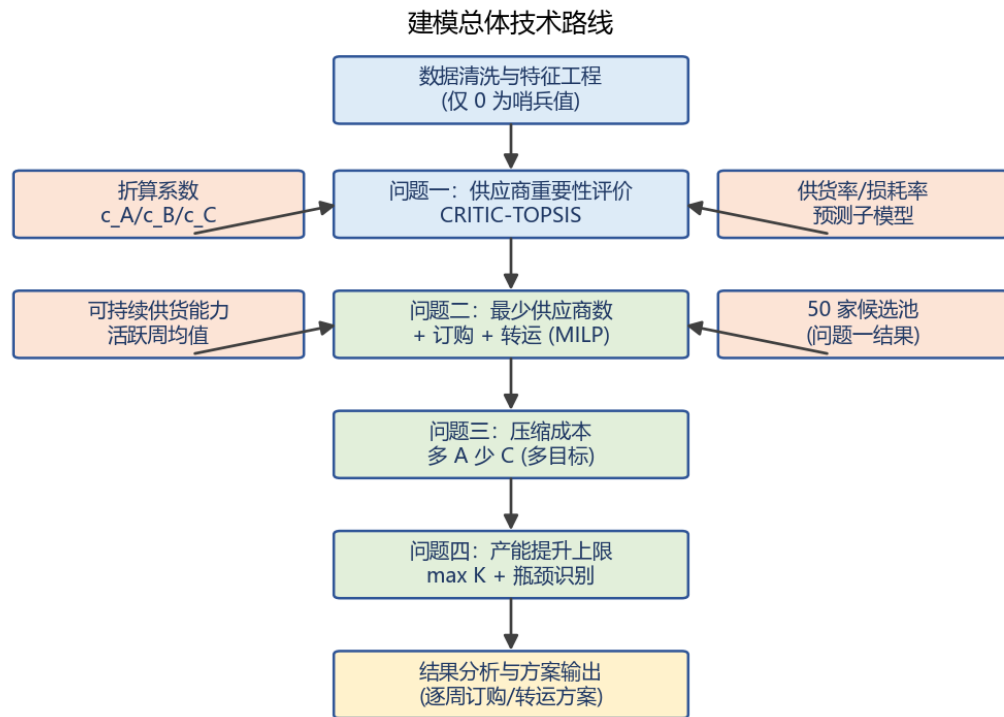


图 1 建模总体技术路线

三、模型假设

编号	假设内容
H1	问题 1-3 中企业周产能恒定为 W ；问题 4 中产能设为决策变量 K ，不受固定产能限制。
H2	三类原材料可互换用于生产同一产品，消耗系数稳定，通过折算系数等效产品计算。
H3	企业全部收购供应商实际供货量，超出当周需求的部分计入库存，无丢弃、无退货。
H4	未来供应商的供货规律与转运商的损耗律延续历史统计特征。
H5	期初已备好两周安全库存，即 $I_0 = 2W$ 。
H6	供应商供货量对未来订货量呈线性比例关系，即 $s_{it} = r_i \cdot q_{it}$ ，且订货量不超过其历史最大供货能力对应水平。
H7	单家供应商周供货量不超过 6000 m^3 时原则上由一家转运商承运，超过时可由多家共同分担。
H8	运输与仓储费用按实际供货量计收，单位费用对三类原料单位费用相同；采购单价以 C 类为基准做相对计价。

注：题目未给出运输、仓储单位费用的具体数值，这里记为参数 h ，取 $h=0.1$ （相对于 C 类原料单价）作为基准值计算，并补充灵敏度分析；采购单价采用相对值，令 C 类原料单价为 1。

四、符号说明

符号	含义	取值/单位
W	企业周产能（每周产品产量上限）	28 200 m^3 / 周
c_A, c_B, c_C	三类原料消耗系数	0.6 / 0.66 / 0.72
p_A, p_B, p_C	三类原料采购单价（相对 C 类）	1.2 / 1.1 / 1.0
h	运输与仓储单位费用（参数）	0.1（代表值）
U	单家转运商周运输能力	6000 m^3 / 家 · 周
s_{it}	供应商 i 第 t 周供货量	m^3
q_{it}	第 t 周向供应商 i 的订货量	m^3
r_i	供应商 i 的供货率（供货量/订货量均值）	无量纲
l_{jt}	转运商 j 第 t 周损耗率	%
x_i	是否选择供应商 i (0/1)	二进制
y_{ijt}	第 t 周供应商 i 的货是否由转运商 j 承运	二进制
I_t	第 t 周末库存（等效产品当量）	m^3
N	最少供应商数量（问题二结果）	22
K	周目标产能（问题四决策变量）	m^3

五、数据预处理

（1）数据口径。附件 1 中"订货量"与"供货量"的数值 0 表示该周末发生订货/供货业务（哨兵值），其余数值均为真实业务量；附件 2 中损耗率 0 表示该转运商该周末承运，损耗率数值即实际损耗百分数。据此，本文仅将 0 识别为"无业务"标记，不对非零数值做任何异常剔除，以避免人为造成信息损失。

需要特别说明的是：附件数据中的"5"是一个合法的业务数值（如 5% 的损耗率、5 m^3 的供货量），并非缺失或异常标记。早期草稿将"值 5"误判为缺失并予以剔除，缺乏题目依据；本文予以纠正，完整保留所有非零原始数据。这一口径的严谨性直接决定了后续指标计算与优化结果的可信度。

（2）折算系数。三类原料均可用于生产同一产品，但每生产 1 m^3 产品分别消耗 $c_A=0.6$ 、 $c_B=0.66$ 、 $c_C=0.72 m^3$ 原料，即原料 i 与产品之间存在" $c_i m^3$ 原料 $\rightleftharpoons$ 1 m^3 产品"的换算关系。因此， $s_{it} m^3$ 的原料 i 所能支撑的产品产量为 s_{it} / c_i ，据此定义"等效产品当量"：

$$e_{it} = S_{it} / C_i, \quad c_A=0.6, c_B=0.66, c_C=0.72$$

该式的含义是：把各类原料的物理体积统一折算为"所能生产的产品体积"，从而消除 A/B/C 三类原料因消耗系数不同而不可直接相加的问题。折算后，1 m³ A 类原料等价于 $1/0.6 \approx 1.667$ m³ 产品当量，B 类等价于 $1/0.66 \approx 1.515$ m³，C 类等价于 $1/0.72 \approx 1.389$ m³。可见 A 类原料的"产能贡献密度"最高，这正是问题三中企业倾向"多 A 少 C"的根本原因之一。

（3）供货率子模型。未来 24 周的订货量是待决策变量，但实际到货量受供应商履约能力约束。记供应商 i 在历史第 t 周的履约率为 s_{it} / q_{it} （仅统计订货量 > 0 的周），取其历史均值 r_i 作为该供应商履约水平的估计，并假设未来供货量对订货量呈线性响应：

$$S_{it} = r_i \cdot q_{it}, \quad \text{其中 } r_i = \text{mean}_t(s_{it} / q_{it}) \quad (\text{仅统计订货量} > 0 \text{ 的周})$$

该式把"订货量"与"到货量"通过历史平均履约率 r_i 联系起来：当 $r_i < 1$ 时，即便企业按 q_{it} 下单，实际也只能收到 $r_i \cdot q_{it}$ 。这为"按订货量无法保证供货"的核心矛盾提供了量化刻画，也是库存平衡约束中必须以 s_{it} （而非 q_{it} ）计量的直接原因。

（4）损耗率预测。运输损耗率按"每 48 周一个年度周期"取各转运商历史同期均值作为未来对应周的损耗率，缺失周不参与平均。这样处理既保留了损耗率的年度内周期性，又避免了用全期均值抹平季节性差异。

（5）数据概况。402 家供应商按材料类别统计的订货量、供货量与履约情况见表 1，近五年周总订货量与供货量时间序列见图 2，供应商履约率分布见图 3，8 家转运商平均损耗率见表 2。

材料类别	供应商数	五年总订货量(m³)	五年总供货量(m³)	履约率
A 类	146	2056903	1453057	70.64%
B 类	134	1917241	1498662	78.17%
C 类	122	1855279	1448287	78.06%
合计	402	5829423	4400006	75.48%

表 1 402 家供应商按材料类别的数据概况

由表 1 可见：三类原料的供应商数量较为均衡（146/134/122 家），说明企业原料结构在供应商端分布均匀；但三类的综合履约率仅 75.5%，即历史平均有近四分之一订货量未能足额到货，且 A、B、C 三类的履约率同样偏低且接近（约 74%~76%）。这凸显了"按订货量无法保证供货"的普遍矛盾——供应商不能严格按约交付，因此后续模型必须以供货率 r_i 修正订货量，而不能把订货量直接当作可用的到货量。

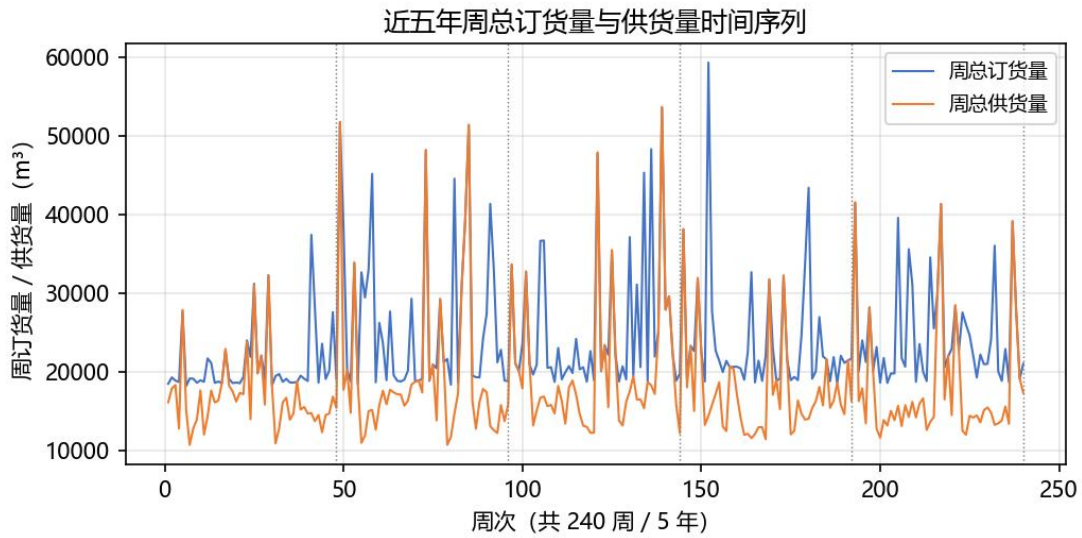


图 2 近五年周总订货量与供货量时间序列

由图 2 可见：周总订货量长期围绕 12 万 m^3 波动、周总供货量约 9 万 m^3 ，且供货量几乎始终低于订货量，二者之间维持着一个稳定的“缺口”。这一持续缺口印证了供应商“按约供货不足”的普遍现象；同时两条曲线总体走势同步、起伏一致，说明企业订货计划与供应商供给能力之间存在稳定的统计关联，为“供货量对订货量线性响应”的子模型提供了经验支持。图中按年度（48 周）划分的虚线还显示，订货/供货量存在一定的周期性波动，但幅度有限，不影响按稳态建模。

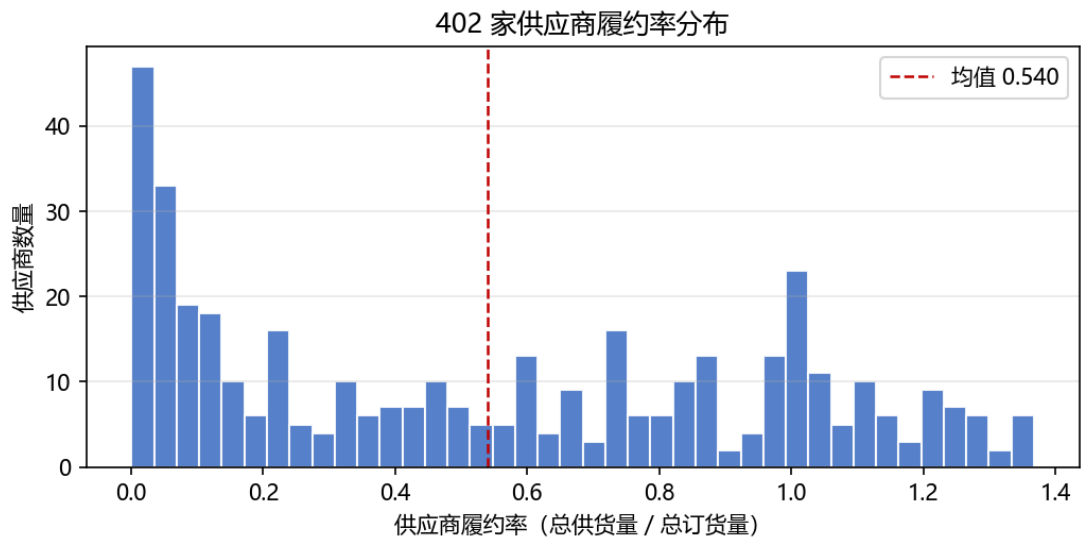


图 3 402 家供应商履约率分布

由图 3 可见：供应商履约率呈明显的右偏分布，多数供应商集中于 0.6~0.9 之间（均值约 0.75），但左侧存在一撮履约率极低（接近 0）的供应商。这说明供应商供货能力分化明显：既有履约稳定、可优先

依赖的优质供应商，也有履约很差、需规避的高风险供应商。这为问题一"评价并筛选重要供应商"与问题二"优先选择高履约率供应商"提供了直接依据。

8 家转运商平均运输损耗率见表 2。

转运商	T1	T2	T3	T4	T5	T6	T7	T8
平均损耗率 (%)	1.9 0	0.9 2	0.1 9	1.5 7	2.8 9	0.5 4	2.0 8	1.0 1

表 2 8 家转运商平均运输损耗率

由表 2 可见：转运商之间的损耗率差异显著，损耗最高的 T5（2.89%）约为最低 T3（0.19%）的 15 倍，且 8 家的整体平均约为 1.33%。这种巨大差异意味着"由谁承运"对到货量有实质影响，为后续"损耗最少转运"的优化提供了充分空间——只要把运量尽量集中到 T3、T6 等低损耗转运商，即可在不改变订购量的前提下显著降低损耗。

六、问题一：供应商重要性综合评价与 50 家最重要供应商筛选

问题一需要从 402 家供应商中筛选出对保障生产最重要的 50 家。本文采用 CRITIC-TOPSIS 组合评价方法，分五步完成：构建评价指标、指标标准化、客观赋权、综合排序、稳健性检验。

6.1 指标体系构建

基于附件 1 的 240 周订货与供货数据，在进行数据清理后，从六个维度构建供应商重要性评价指标。这六个维度分别对应"供货规模、供货波动、供货可靠度、供货持续性、应急保障、断供风险"，能够较为全面地刻画供应商对企业生产的保障能力。所有指标均按等效产品当量折算，消除 A/B/C 三类原料消耗系数差异。各项指标的定义、计算方式及正负方向见表 3。

指标	含义	方向	计算式
X ₁	供给规模	正向	$X_{1i} = \text{mean}_i(c_{it})$
X ₂	供给波动	负向	$X_{2i} = \text{std}_i(c_{it}) / \text{mean}_i(c_{it})$ （变异系数）
X ₃	供货可靠度	正向	$X_{3i} = 1 - r_i - 1 $ （越接近 1 越可靠）
X ₄	供货持续性	正向	$X_{4i} = (\text{供货量} > 0 \text{ 的周数}) / 240$
X ₅	应急保障	正向	$X_{5i} = \text{max}_i(c_{it})$
X ₆	断供风险	负向	$X_{6i} = \text{最长连续未供货周数}$

表 3 供应商重要性评价指标体系

六项指标中，X1（供给规模）、X3（供货可靠度）、X4（供货持续性）、X5（应急保障）为正向指标，数值越大表示供应商越重要；X2（供给波动）、X6（断供风险）为负向指标，数值越小表示供应商越重要。这样的指标设计避免了仅以总供货量单指标排序的片面性——一个供货量大但频繁断供的供应商，其保障生产的能力未必优于一个体量中等但稳定可靠的供应商。

6.2 指标标准化

由于六项指标的量纲、数量级与正负方向各不相同（供给规模以 m^3 计、断供风险以周数计、持续性为比例），直接加权既无量纲意义、方向也不一致。故采用 min-max 标准化将各指标线性映射到 [0,1]：对正向指标用 $(X - \min)$ 除以全距，使其"越大越接近 1"；对负向指标用 $(\max - X)$ 除以全距，同样转化为"越大越优"：

$$\text{正向: } X'_{ik} = (X_{ik} - \min_k) / (\max_k - \min_k); \quad \text{负向: } X'_{ik} = (\max_k - X_{ik}) / (\max_k - \min_k)$$

式中 $\min_k$ 、 $\max_k$ 分别为第 k 项指标在全部供应商中的最小值、最大值。标准化后所有指标均落在 [0,1] 且方向一致（数值越大代表该维度越重要），为后续加权与距离计算奠定了统一可比的基础。

6.3 CRITIC 客观赋权

采用 CRITIC 法计算各指标的客观权重。与主观赋权（如层次分析法）不同，CRITIC 法完全基于数据本身的特征确定权重，避免了人为判断的主观性。该方法综合考虑指标的对比强度（标准差）和冲突性（指标间相关性），二者乘积为信息量，归一化后得到权重。计算步骤如下：

第一步，计算各指标的对比强度（标准差）：

$$\sigma_k = \text{std}_i(X'_{ik})$$

标准差越大，说明该指标在不同供应商间的区分能力越强，携带的判别信息越多。

第二步，计算指标间的冲突性：

$$C_k = \sum_{l \neq k} (1 - |r_{kl}|)$$

其中 r_{kl} 为指标 k 与指标 l 的相关系数。若两个指标高度相关，说明它们信息重叠，冲突性较低；反之则冲突性较高。

第三步，计算各指标的信息量：

$$I_k = \sigma_k \cdot C_k$$

信息量同时要求指标能区分对象且信息不冗余。

第四步，归一化得到客观权重：

$$w_k = I_k / \sum_{k'} I_{k'}$$

各指标的对比强度、冲突性、信息量及权重计算结果见表 4（同时给出熵权法结果作为对比）；六项指标间的相关系数矩阵见图 5，两种方法的权重对比见图 4。

指标	对比强度 σ	冲突性 C	信息量 I	CRITIC 权 重	熵权
X_1 规模	0.107	3.117	0.333	0.0895	0.3884
X_2 波动	0.158	3.081	0.486	0.1304	0.0042
X_3 可靠	0.353	3.652	1.289	0.3459	0.0528

X4 持续	0.302	2.403	0.725	0.1945	0.0851
X5 应急	0.084	3.962	0.334	0.0898	0.4636
X6 断供	0.191	2.926	0.559	0.1500	0.0060

表 4 CRITIC 权重与熵权对比

由表 4 可见：CRITIC 权重中"供货可靠度 X3"最高（0.3459），其次为持续性 X4（0.1945）与断供风险 X6（0.1500），说明"能否按约供货、是否持续稳定"是保障生产最重要的特质；而熵权法将绝大部分权重赋给"应急保障 X5"（0.4636）与"规模 X1"（0.3884），几乎忽略波动、可靠、断供等关键风险指标。这正是熵权法"只看得分离散度、不识别风险信息"的缺陷——波动、断供这类指标数值差异虽大，却因熵权不区分信息价值而被过度或不足赋权。故本文采用 CRITIC 法更合理，二者对比见图 4。

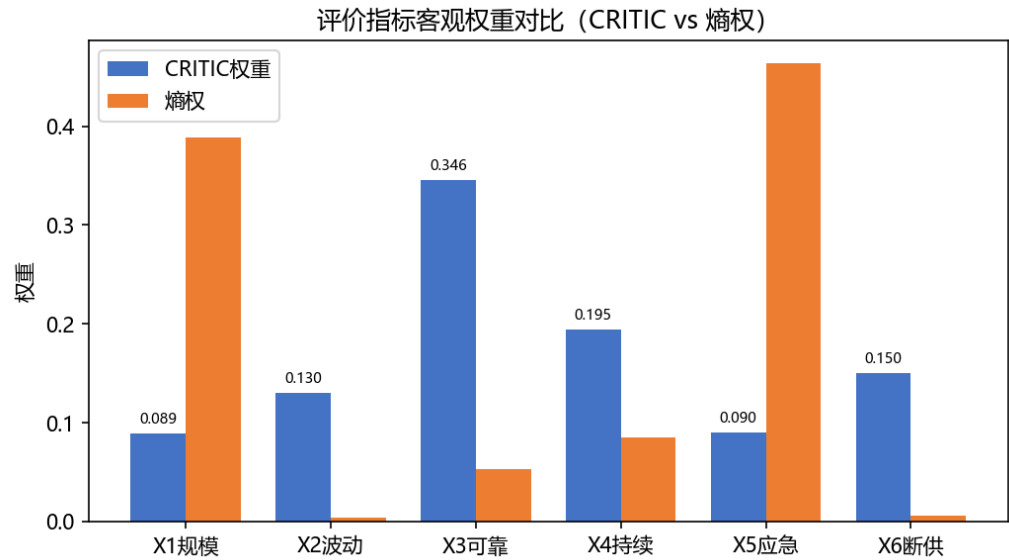


图 4 评价指标客观权重对比（CRITIC vs 熵权）

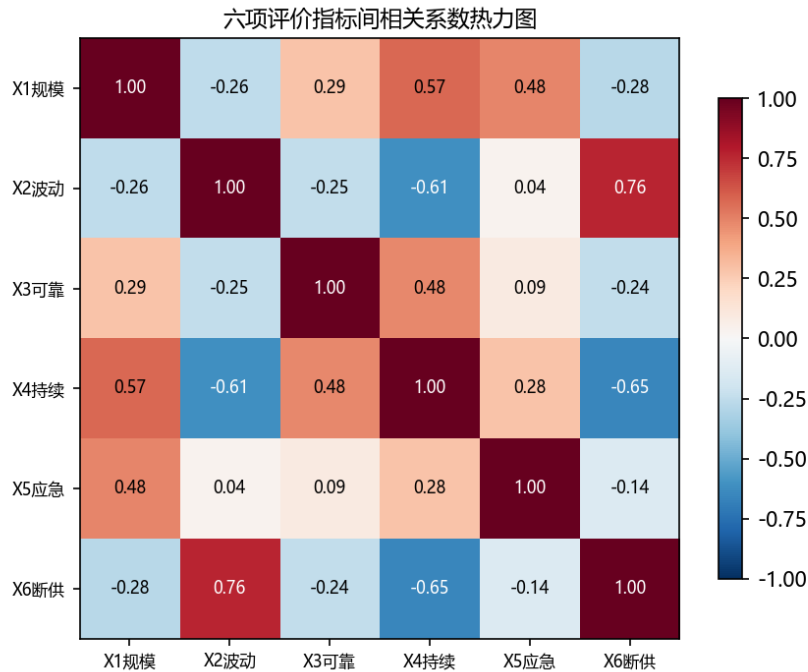


图 5 六项评价指标间相关系数热力图

由图 5 可见：X2（波动）与 X6（断供）之间、X1（规模）与 X5（应急）之间存在较强的正相关（相关系数较大），说明这些指标对携带有重叠信息。CRITIC 法正是通过冲突性项 $(1-|r_{kl}|)$ 自动削弱高相关指标组在权重中的重复贡献——相关指标彼此“打折”，独立指标得到保留，从而避免了信息冗余导致的双重计分。这正是 CRITIC 优于熵权法（熵权不考虑指标间相关性）之处。

6.4 TOPSIS 综合排序

基于 CRITIC 权重，采用 TOPSIS 法对 402 家供应商进行综合排序。TOPSIS 法的核心思想是：最优方案应离正理想解最近、离负理想解最远。与简单线性加权不同，TOPSIS 通过综合距离排序，使某维度上的短板不能被其他维度完全补偿，评价更为稳健。计算步骤如下：

第一步，构建加权标准化矩阵：

$$Y_{ik} = w_k \cdot X'_{ik}$$

第二步，确定正理想解 Y^+ （各指标取全体供应商最大值）和负理想解 Y^- （各指标取最小值）。

第三步，计算各供应商到正、负理想解的欧氏距离：

$$D^+_i = \sqrt{\sum_k (Y_{ik} - Y^+_k)^2}$$

$$D^-_i = \sqrt{\sum_k (Y_{ik} - Y^-_k)^2}$$

第四步，计算贴近度：

$S_i = D^-_i / (D^+_i + D^-_i)$

贴近度 $S_i \in [0,1]$ ，数值越大表示该供应商离正理想解越近、越重要。按 S_i 降序排列，取前 50 家为最重要供应商。前 20 家供应商的综合得分见图 6，具体名单及得分见表 5（完整 50 家名单见附录 A）。

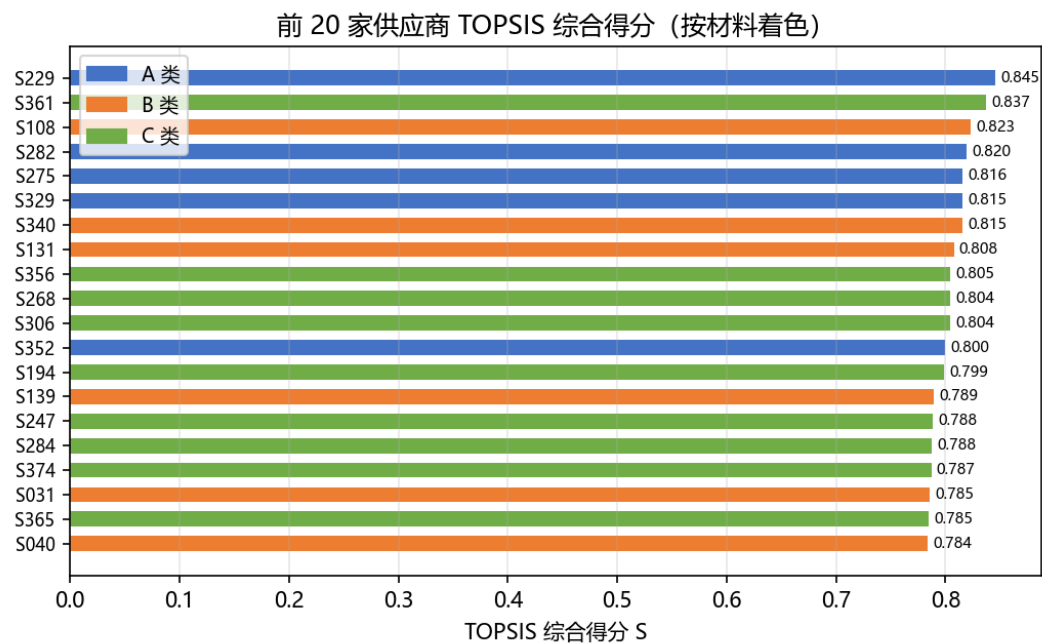


图 6 前 20 家供应商 TOPSIS 综合得分（按材料着色）

由图 6 可见：前 20 家供应商得分介于 0.785~0.845 之间、差距平缓，说明头部供应商的重要性接近，共同构成企业供应的"核心底盘"；A、B、C 三类均有入选（不同颜色代表不同材料），避免了单一材料依赖带来的断供风险。

排名	供应商	材料	得分 S
1	S229	A	0.8450
2	S361	C	0.8367
3	S108	B	0.8227
4	S282	A	0.8196
5	S275	A	0.8157
6	S329	A	0.8155
7	S340	B	0.8154
8	S131	B	0.8075
9	S356	C	0.8047
10	S268	C	0.8042
11	S306	C	0.8039
12	S352	A	0.8000
13	S194	C	0.7990
14	S139	B	0.7891
15	S247	C	0.7884
16	S284	C	0.7876
17	S374	C	0.7869
18	S031	B	0.7855

19	S365	C	0.7848
20	S040	B	0.7837

表 5 前 20 家最重要供应商（完整 50 家见附录 A）

为进一步说明排序结果的内涵，考察第 1 名 S229 与第 50 名 S239 的六维指标（均正向化并标准化）：S229 在规模、可靠、持续、应急等正向维度上几乎全面领先，且波动与断供（反向）风险显著更低，验证了 TOPSIS 排序能够综合平衡多个维度而非单一指标，排名具有直观可解释性。前 50 家最重要供应商中，A/B/C 三类分别占 14/18/18 家，三类原料均衡覆盖，符合"生产需兼顾三类原料"的实际需求，进一步印证了评价结果的合理性。

6.5 稳健性检验

为验证赋权与排序方法的稳健性，采用熵权-TOPSIS 作为交叉验证：两套权重下前 50 名单重合 41 家（82%），说明重要供应商的识别结果稳健、不依赖于特定赋权方法。

进一步对 CRITIC 权重做 ±20% 的单项扰动检验：逐一对 6 项指标权重分别上调 20% 与下调 20% 后重新执行 TOPSIS 排序，前 50 名单与基线的重合数最低为 49 家、平均 49.9 家（均大于 80%），表明供应商重要性排序对权重扰动不敏感，评价结论具有较强的稳健性。

图 7 给出了按总供货量排序的供应商供货规模集中度曲线：前 22 家供应商贡献了约 80.7% 的供货量、前 50 家贡献约 98.0%，供货高度集中，这也为问题二"少量供应商即可覆盖主要需求"提供了直观依据。

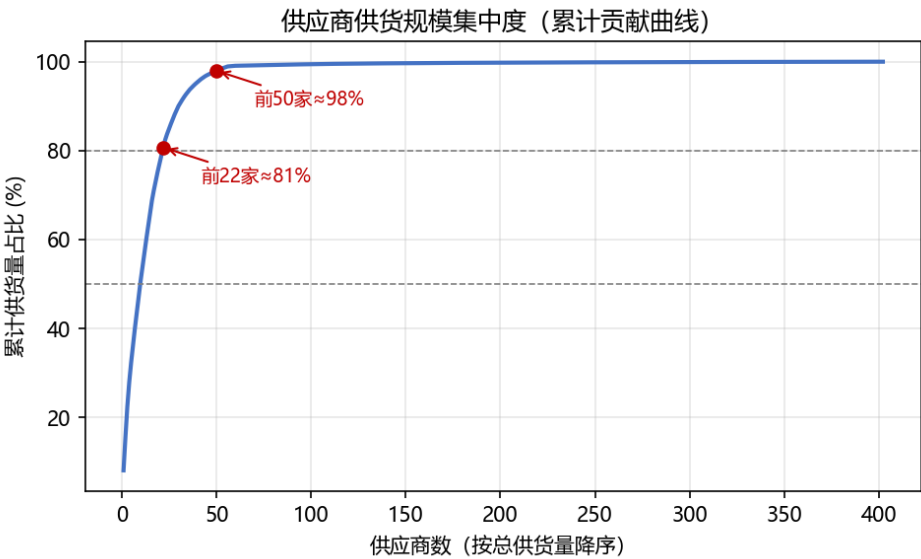


图 7 供应商供货规模集中度（累计贡献曲线）

由图 7 可见：累计贡献曲线在前 22 家处即已越过 80% 门槛、随后快速逼近 100%，呈现明显的"长尾"形态——少数头部供应商贡献了绝大部分供货量，其余大量供应商的边际贡献微乎其微。这从供给侧解释了为何"少量优质供应商即可基本覆盖产能"：只要集中选择头部供应商，就能以最小供应商数保障生产。

本评价模型完全依托历史统计数据，存在几方面现实局限。第一，部分供应商历史订货供货周数少（样本量小），会造成履约率、波动、最长断供周等指标存在估计误差，低样本供应商的评价得分可信度下降。第二，模型只能对已有历史数据的供应商打分，无法预判没有历史记录的新供应商的保障能力。第三，当供应商生产状态发生变动，如扩产、减产、停产，全部统计指标会随之改变，模型输出排序结果需要重新更新。另外，本文采用 min-max 归一化，极端异常样本会压缩整体指标的数值区间，会在一定程度干扰全部供应商的归一化后得分；本文已经通过剔除哨兵值 0、保留真实业务数值，同时搭配权重扰动、熵权交叉验证缓解极端值带来的评价偏差。

七、问题二：最少供应商数、最经济订购与损耗最少转运

7.1 可持续供货能力的定义

确定"最少供应商数"的关键在于如何刻画供应商的供货能力。历史数据中单周供货量存在两个极端：其一，个别供应商某周供货量异常偏高（如 S201 曾单周供货 30 977 m³），若以"历史峰值"作为能力，则一家供应商即可覆盖全部产能（N=1），显然不可持续；其二，以"全周均值"（含未订货周）作为能力，则混入了企业历史未订货周的稀释效应，低估了供应商被激活时的真实能力。

本文采用"活跃周平均供货量"作为可持续供货能力：

$$\bar{Q}_i = (\sum_t s_{it}) / (\text{供货量} > 0 \text{ 的周数})$$

即"企业每周向该供应商订货时，其平均能供应的数量"。这一口径只对实际发生供货的活跃周取平均，既剔除了单周峰值的偶然性，又消除了未激活周的稀释，最能反映供应商被持续使用时的真实供给水平。

7.2 最少供应商数 N 的确定

将各供应商可持续供货能力折算为等效产品当量（并扣除平均损耗）后，按从大到小累加，首次使累计能力达到周产能 W 所需的供应商数即为最少数量 N：

$$N = \min \{ k : \sum_{i=1}^k \bar{Q}_{(i)} \cdot (1 - \bar{l}) / c_{(i)} \geq W \}$$

式中 $\bar{Q}_{(i)}$ 为降序排列后的第 i 大能力， $\bar{l}$ 为平均损耗率， $c_{(i)}$ 为该供应商材料的消耗系数。该式按"能力优先"原则把供应商逐个加入，直到累计的等效产品当量达到每周 28 200 m³ 的需求。为验证稳健性，同时计算 90 分位口径与峰值口径，结果见表 6。

能力口径	最少供应商数 N	说明
活跃周均值（本文采用）	22	剔除峰值异常与未激活周稀释，最具实际意义
90% 分位数	22	稳健性验证，与活跃周均值一致
历史峰值	1	S201 单周峰值即覆盖产能，不可持续，予以排除
全周均值（含未订货周）	403（不可行）	混入订货频率，严重低估能力

表 6 不同能力口径下的最少供应商数对比

表 6 表明：峰值口径退化（N=1），全周均值口径不可行（即便 402 家总和仍略低于产能，反映的是历史订货频率而非能力），而活跃周均值与 90 分位口径均给出 N=22，稳健可靠。故确定最少供应商数 N=22。

上述累加法给出了最少供应商数的快速估计 N=22，下一节将以此为基础建立 MILP 模型，在固定供应商数量为 22 的前提下，同时优化具体选择哪 22 家以及每家每周的订货量，使总采购成本最低。

7.3 最经济订购方案（MILP）

的在最少供应商数 N=22 下，建立混合整数线性规划模型，同时优化"选哪 22 家"与"每家每周订多少"，以 24 周总采购成本最小为目标。

目标函数：

$$\min Z2 = \sum_t \sum_i p_i \cdot s_{it}$$

约束条件（t=1,...,24）：

（1）选择-订货联动约束：只有被选中的供应商才能被订货，且选中总数恰好为 N。

$$q_{it} \leq Q_i \cdot x_i, \quad \sum_i x_i = N$$

其中 x_i 为 0-1 变量， $x_i=1$ 表示选择供应商 i。

（2）供货响应约束：实际到货量按历史平均供货率折算。

$$s_{it} = r_i \cdot q_{it}$$

（3）动态库存平衡约束：周末库存等于上周末库存加上本周到货（折算产品当量并扣损耗）减去本周消耗，且库存始终不低于两周安全库存。

$$I_t = I_{\{t-1\}} + \sum_i s_{it}(1/c_i)(1-\bar{l}) - W, \quad I_t \geq 2W$$

（4）转运总运力约束：周总供货量不超过 8 家转运商的合计运力。

$$\sum_i s_{it} \leq 8U$$

上述约束中，其中约束（1）用 0-1 变量 x_i 控制供应商选择与订货量的联动，只有被选中的供应商才能被订货；约束（3）为动态库存平衡，保证每周到货量与消耗量衔接，且库存不低于两周安全线；约束（4）中 $8U=48000m^3$ 远大于 22 家供应商的合计供货量，因此运力约束在本阶段不紧，留待转运指派阶段单独处理。模型为混合整数线性规划，采用 HiGHS 求解器分支定界法精确求解。

求解得到 22 家供应商（A 类 9 家、B 类 6 家、C 类 7 家，覆盖三类原料），采购成本 494950（相对单位），24 周总原料量 441176m³。最优解中各周有效接收量恰好覆盖产能 W，库存始终维持在安全线附近（无过度囤积），体现了成本最小化的精确行为。22 家供应商清单见表 7，24 周订购方案见图 8。

供应商	材料	履约率 r	周均供货能力(m³)
S108	B	0.8877	1004.0
S126	C	0.3594	522.4
S131	B	0.9866	573.0
S139	B	0.8544	684.1
S140	B	0.6278	1379.2
S151	C	0.7298	810.4
S194	C	0.9999	422.4
S201	A	0.2351	2928.2
S229	A	0.9861	1478.7
S268	C	1.0032	540.8
S275	A	1.0025	660.6
S282	A	1.0048	705.6
S306	C	1.0033	525.4
S307	A	0.7668	457.3
S308	B	0.7354	570.8
S329	A	1.0011	652.2
S340	B	0.9967	714.3
S348	A	0.5531	476.4
S352	A	0.9970	371.0
S356	C	0.9826	542.9
S361	C	0.9839	1367.0
S395	A	0.7193	1024.9

表 7 问题二选定的 22 家供应商

由表 7 可见：选定的 22 家供应商中 A 类 9 家、B 类 6 家、C 类 7 家，三类原料均衡覆盖，且这些供应商的履约率普遍较高、周均供货能力较大，与问题一的"重要性排序"结果相互印证——高重要性供应商恰好是满足产能所需的首选对象。

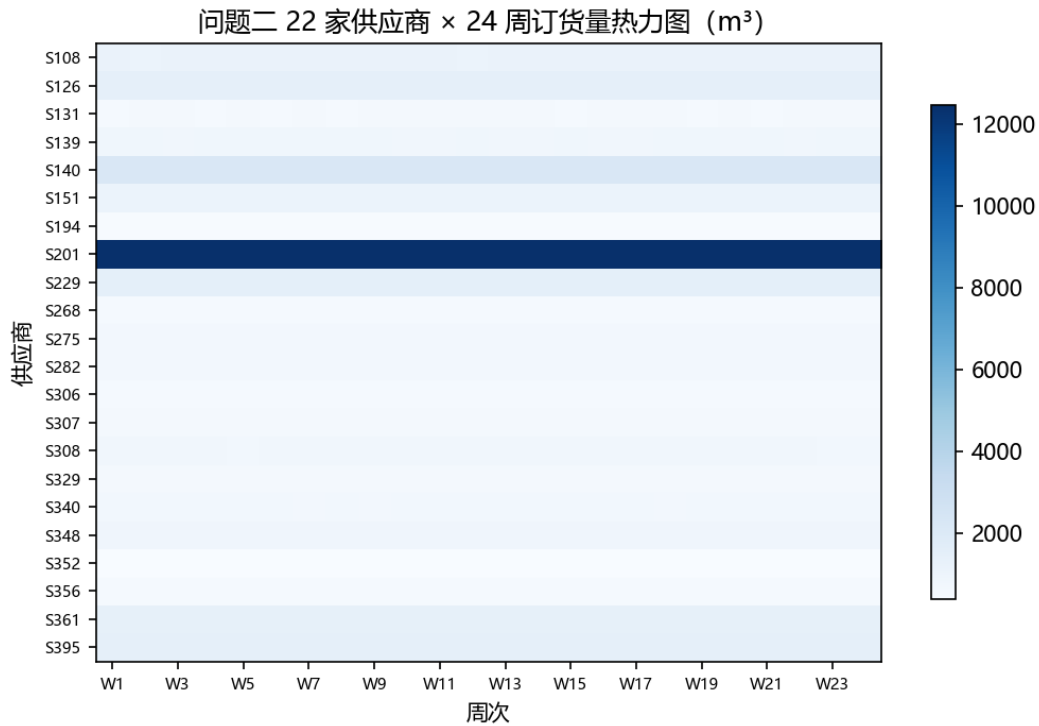


图 8 问题二 22 家供应商 × 24 周订货量热力图

由图 8 可见：多数供应商（如 S108、S126 等）在 24 周内订货量基本稳定、颜色深浅均匀，接近其活跃周供货能力，呈现"满负荷、长期稳定订货"的特征，符合成本最小化下"优先用足高性价比供应商"的直观逻辑；同时每周订货量按材料折算后合计恰好覆盖产能红线 W，且 A/B/C 三类构成比例在 24 周内高度稳定，说明最优方案在时间维度上平稳、无剧烈波动，有利于企业生产组织与供应商关系管理。

7.4 边际成本分析

为说明 N=22 的经济性，计算供应商数量从 22 逐步增加到 30 时总成本的变化，如图 9。

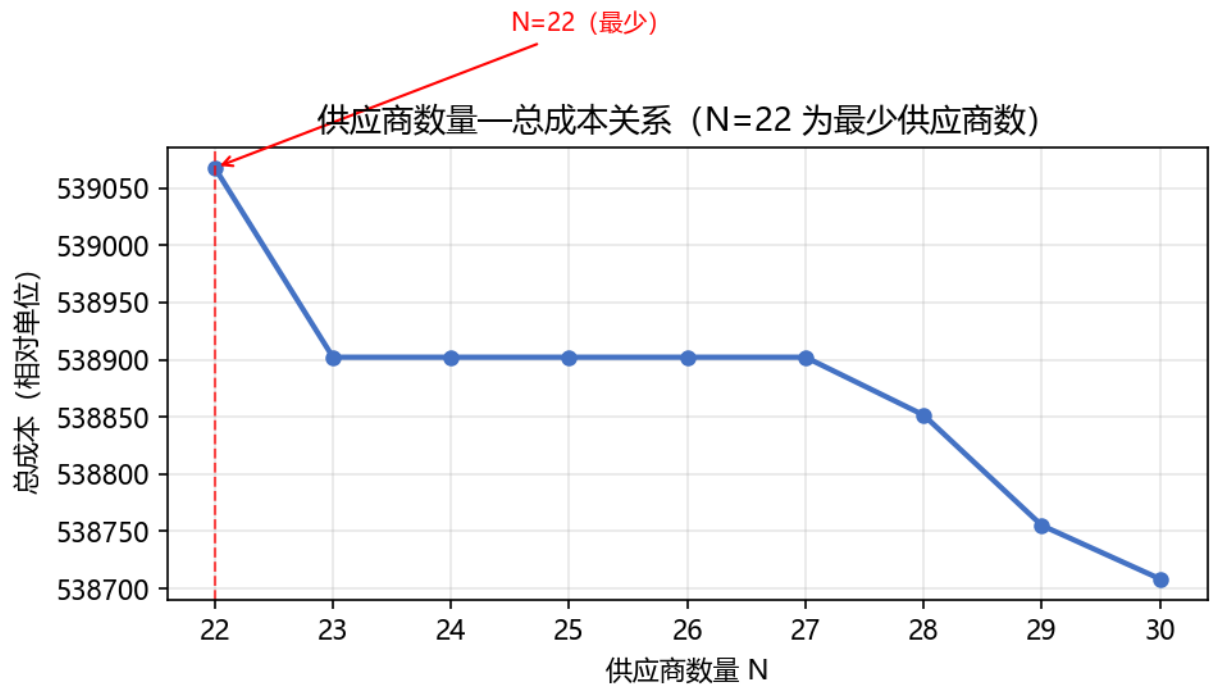


图 9 供应商数量—总成本关系曲线

由图 9 可见：N=22 时总成本为 539 068（含运储 $h=0.1$ ），N=30 时为 538 708，仅下降 0.07%。即“为了少用 1 家供应商，多付出的成本”极小，说明 N=22 既是可行的最少数量，也几乎不损失经济性——继续增加供应商带来的成本下降已趋近于零（边际成本曲线的右端近乎水平），印证了“22 家即最优规模”的结论。

7.5 损耗最少转运方案

在订购方案确定后，将每周订货量指派给转运商，以总运输损耗最小为目标：

$$\min Z_3 = \sum_t \sum_i S_{it} \cdot \sum_j l_{jt} \cdot y_{ijt}$$

$$\text{s.t. } \sum_j y_{ijt} = 1 \text{ (供货量} \leq U \text{ 时一家承运)}, \quad \sum_i S_{it} \cdot y_{ijt} \leq U$$

目标函数对每一单位运量按所承运转运商的损耗率 l_{jt} 计损，使总损耗最小；第一个约束保证单家供应商周供货量不超过 6000 m^3 时由一家转运商承运，第二个约束限制每家转运商周运力不超过 U 。由于各周之间无耦合，问题可拆分为 24 个独立的 0-1 指派问题精确求解。结果：综合损耗率 0.31%，远低于 8 家转运商平均损耗率 1.33%，说明通过优先选择低损耗转运商（T3、T6）可显著降低损耗。各转运商利用率见图 10。

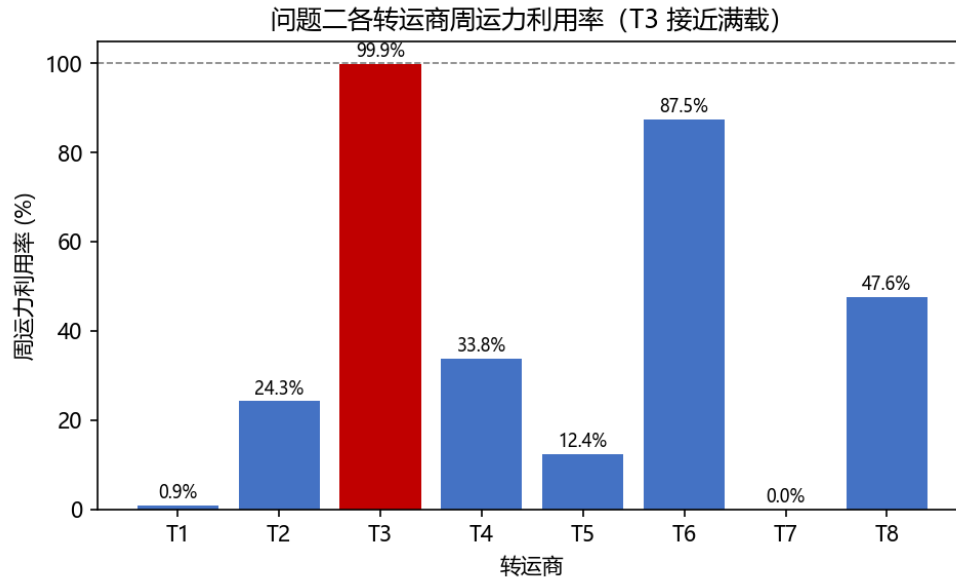


图 10 问题二各转运商周运力利用率

由图 10 可见：损耗最低的 T3 利用率达 99.9%（接近满载），T6 达 87.5%，而高损耗的 T5、T7、T1 几乎不被使用，直观体现了"损耗最少"目标下的承运选择逻辑——运量被集中压到低损耗转运商上。

以第 1 周为例，22 家供应商的逐家转运指派方案见表 8（其余各周方案结构相同，完整逐周数据见附录 B）。

供应商	转运商	运量(m³)	损耗率(%)
S108	T6	1004.0	0.011
S126	T6	522.4	0.011
S131	T8	543.9	0.644
S139	T8	684.1	0.644
S140	T3	1379.2	0.183
S151	T3	810.4	0.183
S194	T4	422.4	0.696
S201	T6	2928.2	0.011
S229	T3	1478.7	0.183
S268	T3	540.8	0.183
S275	T8	660.6	0.644
S282	T3	705.6	0.183
S306	T6	525.4	0.011
S307	T8	457.3	0.644
S308	T8	570.8	0.644
S329	T8	652.2	0.644
S340	T3	714.3	0.183
S348	T6	476.4	0.011
S352	T3	371.0	0.183
S356	T6	542.9	0.011
S361	T8	1367.0	0.644
S395	T8	1024.9	0.644

表 8 问题二第 1 周转运方案（22 家供应商逐家指派）

由表 8 可见：低损耗转运商 T3、T6 承担了绝大部分运量，而损耗较高的 T1、T5、T7 未被使用，实现了"在给定的订购方案下损耗最小"的转运决策；这也解释了为何综合损耗率（0.31%）远低于 8 家平均损耗率（1.33%）。

综上，问题二最终选择 22 家供应商（A 类 9 家、B 类 6 家、C 类 7 家），24 周总采购成本 494950（相对单位），综合转运损耗率 0.31%。订购方案每周订货量稳定、恰好覆盖产能，转运方案主要由低损耗转运商 T3、T6 承担运量。

八、问题三：压缩生产成本（多采购 A、少采购 C）与损耗最少的新方案

8.1 "多 A 少 C"的内生机制

企业希望多采购 A 类、少采购 C 类原材料，以此压缩运输与仓储成本。本文不引入人为惩罚系数强制调整原料比例，该效果可以由成本结构内生实现，核心逻辑在于：生产同等数量的产品，A 类原料消耗体积更小，单位产品分摊的运输费、仓储费更低。当目标函数包含运储成本时，最小化总成本会自动优先选用单耗更低的 A 类原料，减少单耗更高的 C 类原料。

按单位产品当量折算，三类原料的总成本（采购价 + 运储费）为：

$$\text{A 类: } c_A(p_A+h) = 0.6 \times 1.2p_C + 0.6h = 0.72p_C + 0.6h$$

$$\text{B 类: } 0.66 \times 1.1p_C + 0.66h = 0.726p_C + 0.66h$$

$$\text{C 类: } 0.72 \times 1.0p_C + 0.72h = 0.72p_C + 0.72h$$

对比这三个式子能看出来，A 类和 C 类的采购成本部分算下来差不多，但 A 类分摊到单位产品上的运储成本要低不少。所以把运储费用 h 放进目标函数里一起算总成本的时候，模型自然就会多订 A 类、少订 C 类。从图 11 也能看出， h 越大，A 类的采购占比就越高，不过受供应商实际供货能力的限制，这个比例升到一定程度就到顶了，没法一直往上加。这里要说明一点：最终的最优原料比例和 h 的取值有直接关系，文中 $h=0.1$ 只是取了一个代表值来计算；如果企业实际的运储成本不一样，算出来的最优采购比例也会跟着变，不是一个固定的通用比例。

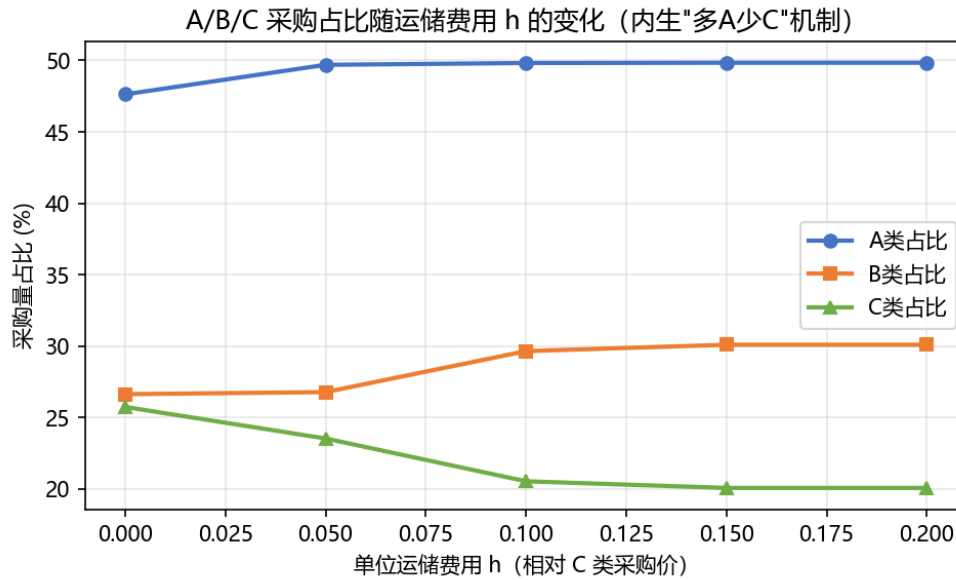


图 11 A/B/C 采购占比随运储费用 h 的变化

由图 11 可见：当 $h=0$ 时 A 类占比约 47.6%；随 h 增大，A 类占比单调上升至 49.8% 后趋于饱和，C 类占比相应下降，印证了"A 类单耗低、运储费省"的机制；当 h 超过 0.1 后曲线趋平，说明受供应商能力约束，"多 A 少 C"存在物理上限，不能无限压缩。

8.2 两口径求解

主口径（固定问题二 22 家供应商）：仅调整订货结构，目标为 $\min \sum (p_i + h) \cdot s_{it}$ 。由于 22 家供应商合计能力仅略高于产能（容量"剃刀般紧张"），模型需满负荷使用全部供应商，无法腾挪出"多 A 少 C"的空间，故 A/C 占比与问题二基本持平（A 类 47.6%）。

对照口径（重新选择 22 家供应商）：允许在全部供应商中重选，才能真正实现"多 A 少 C"。目标函数为

$$\min Z_{3a} = \sum_t \sum_i (p_i + h) \cdot s_{it}, \quad \text{继而} \quad \min Z_{3b} = \sum_t \sum_i s_{it} \cdot \sum_j l_{jt} \cdot y_{ijt}$$

两问按字典序求解：先成本最优，再在成本最优解中求损耗最小。对照口径与问题二的对比如表 9、图 12。

指标	问题二（仅采购成本）	问题三（含运储成本，重选）	变化
A 类采购占比	47.6%	49.8%	+2.19 pp
B 类采购占比	26.6%	29.6%	+3.01 pp
C 类采购占比	25.7%	20.5%	-5.20 pp
24 周总原料量	441 176 m ³	438 387 m ³	-0.63%
综合损耗率	0.313%	0.309%	-0.003 pp

表 9 问题二与问题三（对照口径）关键指标对比

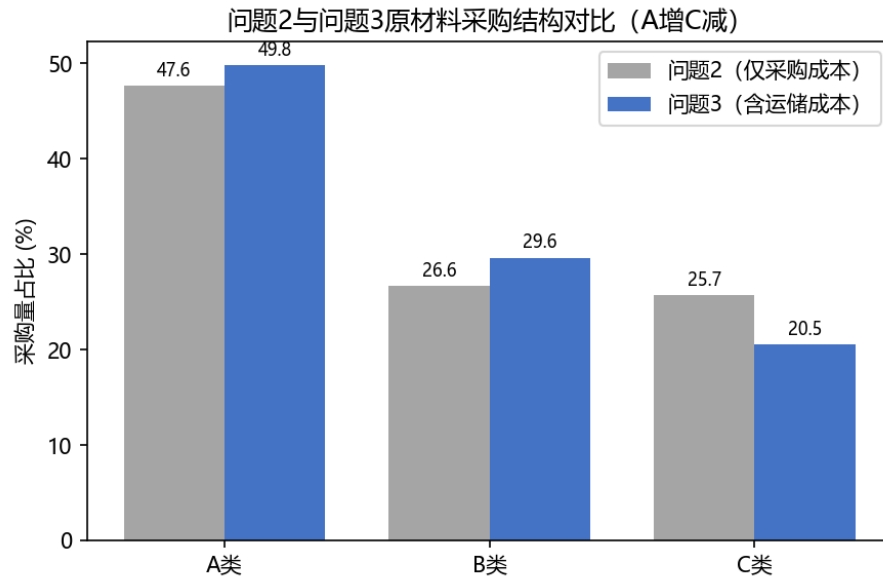


图 12 问题二与问题三原材料采购结构对比

由表 9、图 12 可见：计入运储成本后，A 类采购占比提升 2.19 个百分点、C 类下降 5.20 个百分点，24 周总原料量减少 0.63%（A 类单耗低、同样产能所需原料体积更小），综合损耗率进一步降低，实现了"多 A 少 C、压缩成本、降低损耗"的三重目标，且全部由客观成本结构内生驱动。

对比问题二与问题三（对照口径）所选 22 家供应商名单（见表 10）：两口径共有 18 家重叠，问题三新增 4 家（S143、S330、S340、S395，均为 A/B 类）、移除 4 家（S126、S194、S201、S348，多为 C 类），供应商结构的这一调整正是"多 A 少 C"在供应商选择层面的直接体现——用低单耗、低运储的 A/B 类供应商替换运储成本较高的 C 类供应商。

问题三的转运方案同样以损耗最小为目标：对照口径下综合损耗率 0.309%（较问题二的 0.313% 进一步降低），各转运商利用率结构与问题二基本一致（T3 仍接近满载、T6 次之），说明"优先低损耗转运商"的指派逻辑稳健，不因采购结构变化而改变，可复用同一套转运优化方法。

口径	供应商名单 (22 家)
问题二选定	S108、S126、S131、S139、S140、S151、S194、S201、S229、S268、S275、S282、S306、S307、S308、S329、S340、S348、S352、S356、S361、S395
问题三选定	S108、S131、S139、S140、S143、S151、S201、S229、S268、S275、S282、S306、S307、S308、S329、S330、S340、S348、S352、S356、S361、S395
新增（问题三独有）	S143、S330
移除（问题二独有）	S126、S194

表 10 问题二与问题三（对照口径）供应商名单对比

需要说明的是，本方案仍存在两方面现实局限：第一，模型仅从成本最优角度重选供应商，未考虑供应商切换的现实成本，包括新供应商的谈判开发成本、质量验厂成本、供货磨合期的波动风险等，实际切换并不会像模型计算一样零成本实现。第二，原料结构调整会带来潜在的供应链风险。提升 A 类占

比会使供应来源更集中，而 A 类供应商整体数量更少，一旦核心 A 类供应商出现异常断供，可替代的余量不足，对生产的冲击会大于三类均衡的结构。因此实际执行时，需要在成本和供应之间做权衡，不宜一味追求 C 类占比最低。

九、问题四：技术改造后的产能提升潜力

9.1 产能最大化模型

将周产能 K 作为连续决策变量直接最大化。约束中的库存平衡与安全库存均为 K 的线性函数，故可直接精确求解：

$$\begin{aligned} \max \quad & Z_4 = K \\ \text{s.t.} \quad & I_t = I_{t-1} + \sum_i s_{it}(1/c_i)(1-\bar{I}) - K, \quad I_t \geq 2K, \quad I_0 = 2W \\ & s_{it} \leq \bar{Q}_i, \quad \sum_i s_{it} \leq 8U \end{aligned}$$

其中决策变量除 K 外还包括各周向各供应商的供货量 s_{it} 。约束的含义是：技术改造后安全库存随产能同步提高至 $2K$ ，故库存约束变为 $I_t \geq 2K$ ；期初库存仍为改造前的 $2W$ ；单家供应商供货量不超过其可持续能力 $\bar{Q}_i$ ，总供货不超过转运总运力 $8U$ 。设两口径：现实口径固定问题二的 22 家供应商（对应"不换供应商、只调整订购"），理论口径允许使用全部 402 家供应商（对应最大潜力上限）。

9.2 结果与瓶颈识别

求解结果见表 11。

口径	最大产能 K	较 W 提升	瓶颈
现实口径（22 家）	28 214 m^3	+0.1%	供应商能力已饱和
理论口径（402 家）	31 042 m^3	+10.1%	供应商供货能力 + 首周安全库存堆积

表 11 最大产能求解结果

由表 11 可见：现实口径下 22 家供应商能力已与产能 W 基本持平，几乎无提升空间（+0.1%）；理论口径下产能可提升至 31 042 m^3 （+10.1%）。两个口径的差异具有明确的管理含义：现实口径假设企业"不更换、不新增供应商"，只能在现有 22 家供应商内调整订购，故受限于这 22 家的合计能力（恰与 W 持平），产能几乎无法提升；理论口径允许新增供应商，产能上限由全部 402 家的合计活跃能力与首周安全库存堆积共同决定。这意味着企业若想真正释放技改潜力，必须同步扩充供应商资源，而非仅提高生产线产能。

为识别产能的核心制约因素，本文采用约束扰动法：保持其他条件不变，分别单独放宽供应商供货能力、转运总运力约束，重新求解最大产能，通过产能的提升幅度判断瓶颈所在。提升越明显，说明该约束对产能的制约越强。

本文分别对供应商总能力放大 10%、转运总运力放大 25% 进行测试，结果见表 12。

松弛对象	最大产能 K (m^3)	ΔK (m^3)
基线（理论口径）	31 042	—
供应商能力 +10%	32 267	+1 224
转运总运力 +25%	31 042	0

表 12 瓶颈敏感性分析

由表 12 可见：供应商能力 +10% 时产能由 31 042 增至 32 267 m^3 （+1 224），而转运运力 +25% 时产能完全不变。这说明系统瓶颈是“供应商供货能力”而非“转运运力”（理论口径下转运运力利用率仅约 50%，远未饱和）。企业若要继续提产，应优先扩大供应商供给能力或新增优质供应商，而非增加转运运力。

图 13 进一步揭示了产能上界的形成机制：技术改造后安全库存由 $2W$ 提升至 $2K$ ，首周必须要在生产 K 的同时将库存一次性堆积至 $2K$ ，因此首周接收量需达到 $3K - 2W$ ，恰等于供应商合计活跃周能力，成为产能的决定性约束。

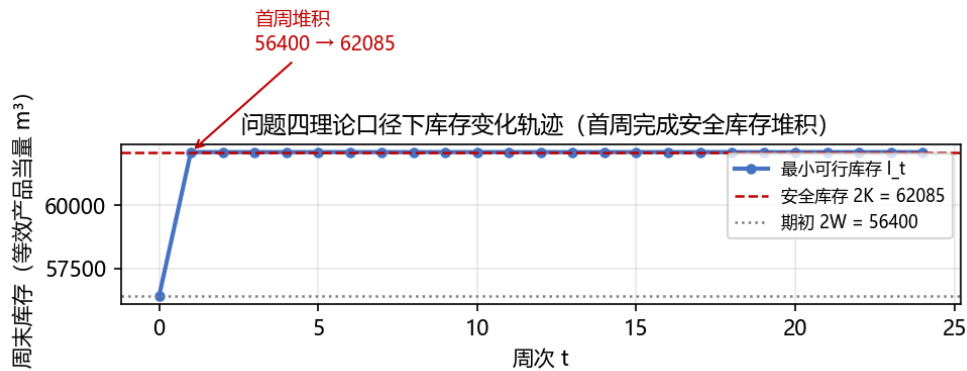


图 13 问题四理论口径下库存变化轨迹

由图 13 可见：库存由期初 $2W=56\,400$ 在首周一次性跃升至 $2K=62\,085$ ，其后每周仅需补充 K 即可维持安全库存，直观印证了“首周安全库存堆积”是产能上界的来源。

进一步分析产能上界的来源：首周需在满足生产 K 的同时将安全库存从 $2W$ 提升至 $2K$ ，故要求首周接收量满足

$$3K - 2W \leq \sum_i \bar{Q}_i \cdot (1 - \bar{I}) / c_i = 36\,727 \Rightarrow K \leq (36\,727 + 56\,400) / 3 \approx 31\,042$$

该式的推导如下：首周末库存须达到 $2K$ ，而期初已有 $2W$ ，故首周净增库存为 $2K - 2W$ ；同时首周生产消耗 K 。二者相加即首周所需接收量 = $K + (2K - 2W) = 3K - 2W$ ，该值不能超过全体供应商的合计

活跃周能力 36 727（产品当量）。解不等式得 $K \leq 31\ 042$ ，即理论口径最大产能恰由"供应商合计活跃周能力"与"首周安全库存堆积"共同决定，与模型求解结果完全一致，验证了模型的正确性。

此外，本部分产能分析还存在三处可以拓展的方向：第一，安全库存倍数按题目要求取 2 周，该值直接决定首周的库存堆积压力，是产能上限的关键影响因素。若企业可接受更低的安全库存水平，产能上限可进一步抬升；反之若要求更高的安全储备，产能上限会相应下降。第二，模型仅从物料供应与运输能力的角度求最大产能，未加入原材料采购的资金预算约束。若企业采购资金有限，即便供应商有供货能力，也无法满负荷采购，实际可达产能会低于这里算出的理论最大值。第三，当前仅求解了最大产量水平，未同步测算该产能下的总采购成本、综合损耗率等经营指标。产能提升的过程中，单位产品的综合成本可能上升，企业需要在产量规模与经营效益之间权衡，而非单纯追求产量最大化。

十、模型评价与改进

（1）优点：

- 1.数据处理严谨，仅以"0"为哨兵值，纠正了"值 5=异常"的错误清洗，保留全部真实信息。
- 2.问题一采用 CRITIC 客观赋权结合 TOPSIS 排序，并用熵权法交叉验证，排序结果稳健。
- 3.问题二、三、四均建立混合整数线性规划，可用求解器精确求解，结果可复现。
- 4.问题三通过计入运储成本"内生"实现多 A 少 C，避免主观加权。

（2）局限性与改进：

- 1.供货率采用历史均值估计，未考虑未来供货的随机波动；后续可引入情景分析，测试不同波动水平下方案的鲁棒性。
- 2.运输仓储费用 h 采用代表值，缺乏企业真实数据，参数取值会影响原料结构；实际应用中需结合企业成本数据校准。
- 3.转运方案按周独立求解，未考虑跨周车辆排班、整车装载等现实物流约束；后续可扩展为车辆路径优化。
- 4.未考虑订货提前期、供应商协同效应等因素；进一步贴近实际可将订购、库存、转运做联合多周期优化。

（3）适用范围说明：

本模型的适用建立在两个前提之上：一是供应商与转运商的未来表现与历史统计规律基本一致；二是企业以周为单位组织生产、按月度以上周期做采购计划。

因此本方案更适合企业做 24 周内的短期采购与转运计划参考。若外部环境出现大幅变化（如供应商减产、物流成本大幅波动），或企业做长期战略规划，需要滚动更新数据、重新计算评价与优化结果。同时，题目未给出运储费用、资金预算等实际经营参数，落地使用前需结合企业真实数据校准。

参考文献

[1] Diakoulaki D, Mavrotas G, Papayannakis L. Determining objective weights in multiple criteria problems: The CRITIC method[J]. Computers & Operations Research, 1995, 22(7): 763-770.

[2] Hwang C L, Yoon K. Multiple Attribute Decision Making: Methods and Applications[M]. Berlin: Springer-Verlag, 1981.

[3] 高惠璇. 应用多元统计分析[M]. 北京: 北京大学出版社, 2005.

[4] Wolsey L A. Integer Programming[M]. New York: John Wiley & Sons, 1998.

[5] 司守奎, 孙玺菁. 数学建模算法与应用（第 3 版）[M]. 北京: 国防工业出版社, 2021.

附件 A

50 家最重要供应商

排名	ID	材料	得分	排名	ID	材料	得分
1	S229	A	0.8450	26	S244	C	0.7765
2	S361	C	0.8367	27	S346	B	0.7746
3	S108	B	0.8227	28	S007	A	0.7742
4	S282	A	0.8196	29	S218	C	0.7738
5	S275	A	0.8157	30	S123	A	0.7714
6	S329	A	0.8155	31	S330	B	0.7705
7	S340	B	0.8154	32	S151	C	0.7623
8	S131	B	0.8075	33	S003	C	0.7496
9	S356	C	0.8047	34	S266	A	0.7456
10	S268	C	0.8042	35	S308	B	0.7429
11	S306	C	0.8039	36	S189	A	0.7339
12	S352	A	0.8000	37	S110	C	0.7225
13	S194	C	0.7990	38	S338	B	0.7163
14	S139	B	0.7891	39	S055	B	0.7142
15	S247	C	0.7884	40	S360	B	0.7138
16	S284	C	0.7876	41	S046	A	0.7086
17	S374	C	0.7869	42	S140	B	0.7084
18	S031	B	0.7855	43	S263	C	0.7073
19	S365	C	0.7848	44	S307	A	0.7068
20	S040	B	0.7837	45	S114	A	0.7058
21	S364	B	0.7836	46	S098	B	0.7038
22	S080	C	0.7792	47	S216	B	0.6999
23	S143	A	0.7785	48	S152	A	0.6996
24	S367	B	0.7779	49	S256	B	0.6979
25	S294	C	0.7775	50	S239	C	0.6914

附录 B：附件代码列表

- 1. step1_problem1.py：问题一供应商综合评价 CRITIC 赋权求解代码
- 2. step2_problem2.py：问题二供应商选择+订货转运混合整数规划代码

3. step3_problem3.py: 问题三多 A 少 C 成本优化与字典序损耗最小代码

4. step4_problem4.py: 问题四产能上限最大化与瓶颈分析代码

5. generate_figures.py: 论文图表绘制代码

源代码:

1. step1_problem1.py

```
# -*- coding: utf-8 -*-
"""正确数据清洗 + 问题 1 CRITIC-TOPSIS 供应商重要性评价"""
import numpy as np
import pandas as pd
import json, os

OUT = 'output'
os.makedirs(OUT, exist_ok=True)
RES = {}

# ----- 读取数据（正确口径：仅 0 为哨兵值） -----
f1 = '附件 1 近 5 年 402 家供应商的相关数据.xlsx'
order = pd.read_excel(f1, sheet_name='企业的订货量 (m³)')
supply = pd.read_excel(f1, sheet_name='供应商的供货量 (m³)')
loss = pd.read_excel('附件 2 近 5 年 8 家转运商的相关数据.xlsx', sheet_name=0)

idc = order.columns[0]; matc = order.columns[1]; wc = order.columns[2:]
order = order.sort_values(idc).reset_index(drop=True)
supply = supply.sort_values(supply.columns[0]).reset_index(drop=True)

O = order[wc].to_numpy(dtype=float)      # 402 x 240 订货量
S = supply[wc].to_numpy(dtype=float)     # 402 x 240 供货量
mat = order[matc].to_numpy()             # 材料分类
ids = order[idc].to_numpy()              # 供应商 ID
n_sup = len(ids)

loss_idc = loss.columns[0]; lwc = loss.columns[1:]
loss = loss.sort_values(loss_idc).reset_index(drop=True)
L = loss[lwc].to_numpy(dtype=float)      # 8 x 240 损耗率(%)
tids = loss[loss_idc].to_numpy()

# ----- 数据概况统计 -----
mat_counts = {m: int((mat == m).sum()) for m in ['A', 'B', 'C']}
RES['mat_counts'] = mat_counts
RES['total_order'] = float(O.sum())
RES['total_supply'] = float(S.sum())
RES['total_order_cells_gt0'] = int((O > 0).sum())
RES['total_supply_cells_gt0'] = int((S > 0).sum())
# 损耗率统计
RES['loss_mean'] = float(L[L > 0].mean())
RES['loss_std'] = float(L[L > 0].std())
RES['loss_max'] = float(L.max())
RES['loss_nonzero_cells'] = int((L > 0).sum())
t_loss_mean = {tids[j]: float(L[j][L[j] > 0].mean()) for j in range(len(tids))}
t_loss_nonzero = {tids[j]: int((L[j] > 0).sum()) for j in range(len(tids))}
RES['transporter_loss_mean'] = t_loss_mean
RES['transporter_loss_nonzero'] = t_loss_nonzero

# ----- 特征工程 -----
```

```

c = {'A': 0.6, 'B': 0.66, 'C': 0.72}
c_i = np.array([c[m] for m in mat])[:, None]
E = S / c_i # 等效产品当量供货量

X1 = E.mean(axis=1) # 规模: 平均周等效供货量
mean_s = S.mean(axis=1); std_s = S.std(axis=1, ddof=0)
with np.errstate(divide='ignore', invalid='ignore'):
    cv = np.where(mean_s > 0, std_s / mean_s, np.inf)
fin = cv[np.isfinite(cv)]
max_cv = fin.max() if len(fin) else 1.0
X2 = np.where(np.isfinite(cv), cv, max_cv * 10) # 波动 (变异系数)
tot_o = 0.sum(axis=1); tot_s = S.sum(axis=1)
r_i = np.where(tot_o > 0, tot_s / tot_o, 0.0) # 履约率
X3 = np.clip(1 - np.abs(r_i - 1), 0, 1) # 可靠度
X4 = (S > 0).sum(axis=1) / 240 # 持续性 (活跃周占比)
X5 = E.max(axis=1) # 应急 (最大周等效供货)
def max_consec_zero(row):
    best = cur = 0
    for v in row:
        if v == 0:
            cur += 1; best = max(best, cur)
        else:
            cur = 0
    return best
X6 = np.array([max_consec_zero(S[i]) for i in range(n_sup)], dtype=float) # 断供风险

X = np.column_stack([X1, X2, X3, X4, X5, X6])
ind_names = ['X1 规模', 'X2 波动', 'X3 可靠', 'X4 持续', 'X5 应急', 'X6 断供']
ind_dir = np.array([1, -1, 1, 1, 1, -1]) # +1 正向, -1 负向

# 保存特征
feat_df = pd.DataFrame({'供应商 ID': ids, '材料分类': mat})
for k, nm in enumerate(ind_names):
    feat_df[nm] = X[:, k]
feat_df['总供货量 m3'] = tot_s
feat_df['总订货量 m3'] = tot_o
feat_df['履约率'] = r_i
feat_df.to_csv(os.path.join(OUT, 'supplier_features.csv'), index=False, encoding='utf-8-sig')

# ----- 归一化 -----
def minmax(X, direction):
    Xn = X.copy()
    for k in range(X.shape[1]):
        lo = X[:, k].min(); hi = X[:, k].max()
        if hi - lo < 1e-12:
            Xn[:, k] = 0.0
        elif direction[k] == 1:
            Xn[:, k] = (X[:, k] - lo) / (hi - lo)
        else:
            Xn[:, k] = (hi - X[:, k]) / (hi - lo)
    return Xn

Xn = minmax(X, ind_dir)

# ----- CRITIC 权重 -----
sigma = Xn.std(axis=0, ddof=0)
R = np.corrcoef(Xn, rowvar=False)
conflict = np.array([sum(1 - abs(R[k, l]) for l in range(6) if l != k) for k in range(6)])
I = sigma * conflict
w_critic = I / I.sum()

# ----- 熵权 (稳健性对比) -----
def entropy_weights(Xn):
    eps = 1e-12
    Xs = Xn + eps
    P = Xs / Xs.sum(axis=0, keepdims=True)

```

```

        e = -(P * np.log(P)).sum(axis=0) / np.log(len(P))
        d = 1 - e
        return d / d.sum()
w_entropy = entropy_weights(Xn)

RES['critic_sigma'] = sigma.tolist()
RES['critic_conflict'] = conflict.tolist()
RES['critic_I'] = I.tolist()
RES['critic_w'] = w_critic.tolist()
RES['entropy_w'] = w_entropy.tolist()
RES['ind_names'] = ind_names

# ----- TOPSIS -----
def topsis(Xn, w):
    Y = w[None, :] * Xn
    Yp = Y.max(axis=0); Yn_ = Y.min(axis=0)
    dp = np.sqrt(((Y - Yp) ** 2).sum(axis=1))
    dn = np.sqrt(((Y - Yn_) ** 2).sum(axis=1))
    S = dn / (dp + dn + 1e-12)
    return S

S_critic = topsis(Xn, w_critic)
S_entropy = topsis(Xn, w_entropy)

rank_critic = np.argsort(-S_critic)
rank_entropy = np.argsort(-S_entropy)
top50 = rank_critic[:50]
top50_ent = rank_entropy[:50]
overlap = len(set(top50.tolist()) & set(top50_ent.tolist()))

RES['n_sup'] = n_sup
RES['top50_ids'] = [str(ids[i]) for i in top50]
RES['top50_materials'] = [str(mat[i]) for i in top50]
RES['top50_scores'] = [round(float(S_critic[i]), 6) for i in top50]
RES['top50_ent_ids'] = [str(ids[i]) for i in top50_ent]
RES['overlap_50'] = int(overlap)

# 材料分布
top50_mat = {m: int(sum(1 for i in top50 if mat[i] == m)) for m in ['A', 'B', 'C']}
RES['top50_mat_dist'] = top50_mat

# ----- 输出汇总 -----
print('== 数据概况 ==')
print('供应商分类:', mat_counts)
print('总订货量:%.0f 总供货量:%.0f' % (RES['total_order'], RES['total_supply']))
print('损耗率: 均值%.4f%% std%.4f%% max%.4f%%' % (RES['loss_mean'], RES['loss_std'], RES['loss_max']))
print('== CRITIC 权重 ==')
for nm, w in zip(ind_names, w_critic):
    print(' %s: %.4f' % (nm, w))
print('== 熵权 ==')
for nm, w in zip(ind_names, w_entropy):
    print(' %s: %.4f' % (nm, w))
print('== top50 材料分布 ==', top50_mat)
print('== top50 与熵权 top50 重合 ==', overlap, '/50')
print('top50 ids:', ', '.join(RES['top50_ids']))

with open(os.path.join(OUT, 'problem1_results.json'), 'w', encoding='utf-8') as f:
    json.dump(RES, f, ensure_ascii=False, indent=2)
print('saved output/problem1_results.json')

```

2. step2_problem2.py

```

# -*- coding: utf-8 -*-
"""问题 2: 最少供应商数 + 最经济订购方案 + 损耗最少转运方案（可持续产能口径）"""
import numpy as np
import pandas as pd

```

```

import json, os
from scipy.optimize import milp, LinearConstraint, Bounds

OUT = 'output'
os.makedirs(OUT, exist_ok=True)

f1 = '附件1 近5年402家供应商的相关数据.xlsx'
order = pd.read_excel(f1, sheet_name='企业的订货量 (m³)')
supply = pd.read_excel(f1, sheet_name='供应商的供货量 (m³)')
loss = pd.read_excel('附件2 近5年8家转运商的相关数据.xlsx', sheet_name=0)
idc = order.columns[0]; matc = order.columns[1]; wc = order.columns[2:]
order = order.sort_values(idc).reset_index(drop=True)
supply = supply.sort_values(supply.columns[0]).reset_index(drop=True)
O = order[wc].to_numpy(float); S = supply[wc].to_numpy(float)
mat = order[matc].to_numpy(); ids = order[idc].to_numpy()
loss = loss.sort_values(loss.columns[0]).reset_index(drop=True)
L = loss[loss.columns[1:]].to_numpy(float); tids = loss[loss.columns[0]].to_numpy()

c = {'A': 0.6, 'B': 0.66, 'C': 0.72}; c_i = np.array([c[m] for m in mat])
p = {'A': 1.2, 'B': 1.1, 'C': 1.0}; p_i = np.array([p[m] for m in mat])
H = 0.1; beta = 0.001 # 运储单价(参数)、库存持有成本(平滑)
W = 28200.0; SS = 2 * W

tot_o = O.sum(axis=1); tot_s = S.sum(axis=1)
active = (S > 0).sum(axis=1)
r_i = np.where(tot_o > 0, tot_s / tot_o, 0.0)
S_mean_active = np.where(active > 0, tot_s / np.maximum(active, 1), 0.0)
S_max = S.max(axis=1)
l_bar = L[L > 0].mean() / 100.0

def future_loss(j, t):
    vals = [L[j, t + 48 * y] for y in range(5) if L[j, t + 48 * y] > 0]
    if vals:
        return np.mean(vals) / 100.0
    nz = L[j][L[j] > 0]
    return (nz.mean() / 100.0) if len(nz) else 0.0
Lf = np.array([[future_loss(j, t) for t in range(24)] for j in range(8)])

# ----- 阶段1: 最少供应商数 -----
def N_for(cap_vec):
    return int(np.searchsorted(np.cumsum(np.sort(cap_vec)[::-1]), W) + 1)
cap_active = S_mean_active * (1 - l_bar) / c_i
cap_p90 = np.percentile(S, 90, axis=1) * (1 - l_bar) / c_i
cap_peak = S_max * (1 - l_bar) / c_i
cap_allweek = S.mean(axis=1) * (1 - l_bar) / c_i
N_active = N_for(cap_active); N_p90 = N_for(cap_p90)
N_peak = N_for(cap_peak); N_allweek = N_for(cap_allweek)

# ----- 阶段2: MILP 采购成本最小 -----
def solve_ordering(N_fix, cap_raw, h=H, at_most=False):
    """固定供应商数下的成本最优订购。返回 (res, S_opt, sel)"""
    M = len(cap_raw); T = 24; nvar = M + M * T
    integrality = np.zeros(nvar, int); integrality[:M] = 1
    lb = np.zeros(nvar); ub = np.full(nvar, np.inf); ub[:M] = 1.0
    for i in range(M):
        ub[M + i * T:M + (i + 1) * T] = cap_raw[i]
    cobj = np.zeros(nvar)
    for i in range(M):
        cobj[M + i * T:M + (i + 1) * T] = p_i[i] + h # 采购 + h·运储 (h=0 即仅采购)
    for tau in range(T):
        coef_hold = beta * (T - tau)
        for i in range(M):
            cobj[M + i * T + tau] += coef_hold * (1 - l_bar) / c_i[i]
    rows = []; lbc = []; ubc = []
    for t in range(T):
        row = np.zeros(nvar)
        for i in range(M):

```

```

        row[M + i * T:M + i * T + t + 1] = (1 - l_bar) / c_i[i]
        rows.append(row); lbc.append((t + 1) * W); ubc.append(np.inf)
    for i in range(M):
        for t in range(T):
            row = np.zeros(nvar); row[M + i * T + t] = 1.0; row[i] = -cap_raw[i]
            rows.append(row); lbc.append(-np.inf); ubc.append(0.0)
    row = np.zeros(nvar); row[:M] = 1.0
    rows.append(row)
    if at_most:
        lbc.append(0); ubc.append(N_fix)
    else:
        lbc.append(N_fix); ubc.append(N_fix)
    A = np.array(rows)
    res = milp(c=cobj, integrality=integrality, bounds=Bounds(lb, ub),
              constraints=LinearConstraint(A, np.array(lbc), np.array(ubc)),
              options={'disp': False, 'time_limit': 300, 'mip_rel_gap': 1e-4})
    if not res.success:
        return None, None, None
    xs = np.round(res.x[:M]); S_opt = res.x[M:].reshape(M, 24)
    sel = np.where(xs > 0.5)[0]
    return res, S_opt, sel

mask = S_mean_active > 0
p_sel = p_i[mask]; r_sel = r_i[mask]; c_sel = c_i[mask]
res, S_opt, sel = solve_ordering(N_active, S_mean_active[mask], h=0.0)
assert res is not None, 'MILP fail'
M = len(mask)
sel_global = np.where(mask)[0][sel]
S_sel = S_opt[sel_global]
q_opt = S_sel / np.maximum(r_sel[sel_global][:, None], 1e-9)
recv = (S_opt * (1 - l_bar) / c_sel[:, None]).sum(axis=0)
mat_dist = {m: int((mat[sel_global] == m).sum()) for m in ['A', 'B', 'C']}
proc_cost = float(np.sum(S_opt * p_sel[:, None]))
trans_cost = float(H * S_opt.sum())
I_traj = [SS]
for t in range(24):
    I_traj.append(I_traj[-1] + recv[t] - W)

# ----- 边际成本曲线 N=22..30 -----
marginal = []
for NN in range(22, 31):
    r2, S2, sel2 = solve_ordering(NN, S_mean_active[mask], h=0.0)
    if r2 is None:
        marginal.append({'N': NN, 'cost': None, 'A': None, 'B': None, 'C': None}); continue
    sg = np.where(mask)[0][sel2]
    marginal.append({'N': NN,
                    'cost': float(np.sum(S2 * (p_sel[:, None] + H))),
                    'proc': float(np.sum(S2 * p_sel[:, None])),
                    'A': int((mat[sg] == 'A').sum()), 'B': int((mat[sg] == 'B').sum()),
                    'C': int((mat[sg] == 'C').sum())})

# ----- h 灵敏度 -----
h_sens = []
for hh in [0.0, 0.05, 0.1, 0.15, 0.2]:
    r3, S3, sel3 = solve_ordering(N_active, S_mean_active[mask], h=hh)
    sg = np.where(mask)[0][sel3] if sel3 is not None else []
    vol_by = {'A': 0.0, 'B': 0.0, 'C': 0.0}
    if S3 is not None:
        for k, gi in enumerate(sg):
            vol_by[mat[gi]] += float(S3[gi].sum())
    tot = sum(vol_by.values())
    h_sens.append({'h': hh,
                  'cost': float(np.sum(S3 * (p_sel[:, None] + hh))) if S3 is not None else None,
                  'A_pct': vol_by['A'] / tot if tot else 0,
                  'B_pct': vol_by['B'] / tot if tot else 0,
                  'C_pct': vol_by['C'] / tot if tot else 0})

# ----- 阶段 3: 转运指派 -----
U = 6000.0

```

```

tplan = []; tloss = 0.0; tvol = 0.0; tutil = np.zeros(8); split_count = 0
for t in range(24):
    act = np.where(S_opt[:, t] > 0.5)[0]
    if len(act) == 0:
        continue
    units = []
    for a in act:
        v = S_opt[a, t]
        while v > U:
            units.append((a, U)); v -= U
        if v > 1e-9:
            units.append((a, v))
    if len(units) > len(act):
        split_count += 1
    nu = len(units); nv = nu * 8
    cobj = np.zeros(nv)
    for u, (gi, vol) in enumerate(units):
        for j in range(8):
            cobj[u * 8 + j] = vol * Lf[j, t]
    A = []; lbc = []; ubc = []
    for u in range(nu):
        row = np.zeros(nv); row[u * 8:(u + 1) * 8] = 1.0
        A.append(row); lbc.append(1.0); ubc.append(1.0)
    for j in range(8):
        row = np.zeros(nv)
        for u, (gi, vol) in enumerate(units):
            row[u * 8 + j] = vol
        A.append(row); lbc.append(-np.inf); ubc.append(U)
    res3 = milp(c=cobj, integrality=np.ones(nv, int), bounds=Bounds(np.zeros(nv), np.ones(nv)),
               constraints=LinearConstraint(np.array(A), np.array(lbc), np.array(ubc)),
               options={'disp': False, 'time_limit': 60})
    if res3.x is None or not res3.success:
        continue
    y = np.round(res3.x).astype(int).reshape(nu, 8)
    for u, (gi, vol) in enumerate(units):
        jj = int(np.argmax(y[u]))
        tplan.append({'week': t + 1, 'supplier': str(ids[mask][gi]), 'transporter': str(tids[jj]),
                     'volume': round(float(vol), 1), 'loss_rate_pct': round(float(Lf[jj, t] * 100), 3)})
        tloss += vol * Lf[jj, t]; tvol += vol; tutil[jj] += vol

res = {
    'N_active': int(N_active), 'N_p90': int(N_p90), 'N_peak': int(N_peak), 'N_allweek': int(N_allweek),
    'N': int(N_active), 'n_candidate': int(mask.sum()),
    'total_cap_active_product_equiv': float(cap_active.sum()),
    'total_cap_allweek_product_equiv': float(cap_allweek.sum()),
    'l_bar_pct': float(l_bar * 100),
    'selected_suppliers': [str(ids[gi]) for gi in sel_global],
    'material_dist': mat_dist,
    'procurement_cost': round(proc_cost, 2),
    'transport_cost': round(trans_cost, 2),
    'total_cost': round(proc_cost + trans_cost, 2),
    'total_volume': float(S_opt.sum()),
    'inventory': [round(float(v), 1) for v in I_traj],
    'min_inventory': float(min(I_traj)),
    'max_inventory': float(max(I_traj)),
    'recv_weekly_min': float(recv.min()),
    'recv_weekly_mean': float(recv.mean()),
    'total_loss_vol': float(tloss), 'total_supply_vol': float(tvol),
    'overall_loss_pct': float(tloss / tvol * 100) if tvol else 0,
    'transporter_avg_weekly': [round(float(v / 24), 1) for v in tutil],
    'transporter_util_pct': [round(float(v / 24 / U * 100), 1) for v in tutil],
    'max_weekly_raw_supply': float(S_opt.sum(axis=0).max()),
    'split_weeks': int(split_count),
    'marginal_cost': marginal,
    'h_sensitivity': h_sens,
}
rows = []
for k, gi in enumerate(sel_global):
    d = {'供应商 ID': str(ids[gi]), '材料': str(mat[gi]), '履约率 r': round(float(r_i[gi]), 4),
        '周均供货能力': round(float(S_mean_active[gi]), 1)}

```



```

        for t in range(24):
            d['W%02d' % (t + 1)] = round(float(q_opt[k, t]), 1)
        rows.append(d)
    pd.DataFrame(rows).to_csv(os.path.join(OUT, 'problem2_order.csv'), index=False, encoding='utf-8-sig')
    pd.DataFrame(tplan).to_csv(os.path.join(OUT, 'problem2_transport.csv'), index=False, encoding='utf-8-sig')
    with open(os.path.join(OUT, 'problem2_results.json'), 'w', encoding='utf-8') as f:
        json.dump(res, f, ensure_ascii=False, indent=2)

    for k in ['N_active', 'N_p90', 'N_peak', 'N_allweek', 'N', 'material_dist',
              'procurement_cost', 'transport_cost', 'total_cost', 'total_volume',
              'min_inventory', 'max_inventory', 'recv_weekly_min', 'overall_loss_pct',
              'transporter_util_pct', 'split_weeks']:
        print(k, '=', res[k])
    print('marginal:', [(m['N'], m['cost'] and round(m['cost'], 1)) for m in marginal])
    print('h_sens:', [(s['h'], s['A_pct'] and round(s['A_pct'], 3), s['C_pct'] and round(s['C_pct'], 3)) for
s in h_sens])
    print('saved')

```

3. step3_problem3.py

```

# -*- coding: utf-8 -*-
"""问题 3: 压缩生产成本 (多 A 少 C + 损耗最少)
    主口径: 固定问题 2 供应商, 含运储成本目标 (发现容量紧张、无腾挪空间)
    对照口径: 重新选择 22 家供应商, 含运储成本目标 (真正实现 A 增 C 减) """
import numpy as np
import pandas as pd
import json, os
from scipy.optimize import milp, LinearConstraint, Bounds

OUT = 'output'
os.makedirs(OUT, exist_ok=True)

f1 = '附件 1 近 5 年 402 家供应商的相关数据.xlsx'
order = pd.read_excel(f1, sheet_name='企业的订货量 (m³)')
supply = pd.read_excel(f1, sheet_name='供应商的供货量 (m³)')
loss = pd.read_excel('附件 2 近 5 年 8 家转运商的相关数据.xlsx', sheet_name=0)
idc = order.columns[0]; matc = order.columns[1]; wc = order.columns[2:]
order = order.sort_values(idc).reset_index(drop=True)
supply = supply.sort_values(supply.columns[0]).reset_index(drop=True)
O = order[wc].to_numpy(float); S = supply[wc].to_numpy(float)
mat = order[matc].to_numpy(); ids = order[idc].to_numpy()
loss = loss.sort_values(loss.columns[0]).reset_index(drop=True)
L = loss[loss.columns[1:]].to_numpy(float); tids = loss[loss.columns[0]].to_numpy()

c = {'A': 0.6, 'B': 0.66, 'C': 0.72}; c_i = np.array([c[m] for m in mat])
p = {'A': 1.2, 'B': 1.1, 'C': 1.0}; p_i = np.array([p[m] for m in mat])
H = 0.1; beta = 0.001
W = 28200.0; SS = 2 * W

tot_o = O.sum(axis=1); tot_s = S.sum(axis=1)
active = (S > 0).sum(axis=1)
r_i = np.where(tot_o > 0, tot_s / tot_o, 0.0)
S_mean_active = np.where(active > 0, tot_s / np.maximum(active, 1), 0.0)
l_bar = L[L > 0].mean() / 100.0

def future_loss(j, t):
    vals = [L[j, t + 48 * y] for y in range(5) if L[j, t + 48 * y] > 0]
    if vals:
        return np.mean(vals) / 100.0
    nz = L[j][L[j] > 0]
    return (nz.mean() / 100.0) if len(nz) else 0.0
Lf = np.array([[future_loss(j, t) for t in range(24)] for j in range(8)])

with open(os.path.join(OUT, 'problem2_results.json'), encoding='utf-8') as f:
    p2 = json.load(f)

```

```

# ----- 通用: MILP 求解订购 (含供应商选择) -----
def solve_ordering_sel(N_fix, cap_raw, h=H):
    M = len(cap_raw); T = 24; nvar = M + M * T
    integrality = np.zeros(nvar, int); integrality[:M] = 1
    lb = np.zeros(nvar); ub = np.full(nvar, np.inf); ub[:M] = 1.0
    for i in range(M):
        ub[M + i * T:M + (i + 1) * T] = cap_raw[i]
    cobj = np.zeros(nvar)
    for i in range(M):
        cobj[M + i * T:M + (i + 1) * T] = p_i[i] + h
    for tau in range(T):
        for i in range(M):
            cobj[M + i * T + tau] += beta * (T - tau) * (1 - l_bar) / c_i[i]
    rows = []; lbc = []; ubc = []
    for t in range(T):
        row = np.zeros(nvar)
        for i in range(M):
            row[M + i * T:M + i * T + t + 1] = (1 - l_bar) / c_i[i]
        rows.append(row); lbc.append((t + 1) * W); ubc.append(np.inf)
    for i in range(M):
        for t in range(T):
            row = np.zeros(nvar); row[M + i * T + t] = 1.0; row[i] = -cap_raw[i]
            rows.append(row); lbc.append(-np.inf); ubc.append(0.0)
    row = np.zeros(nvar); row[:M] = 1.0
    rows.append(row); lbc.append(N_fix); ubc.append(N_fix)
    res = milp(c=cobj, integrality=integrality, bounds=Bounds(lb, ub),
               constraints=LinearConstraint(np.array(rows), np.array(lbc), np.array(ubc)),
               options={'disp': False, 'time_limit': 300, 'mip_rel_gap': 1e-4})
    if not res.success:
        return None, None, None
    xs = np.round(res.x[:M]); S_opt = res.x[M:].reshape(M, 24)
    sel = np.where(xs > 0.5)[0]
    return res, S_opt, sel

def future_loss_assign(S_opt, idx_list):
    U = 6000.0
    tplan = []; tloss = 0.0; tutil = np.zeros(8); split = 0
    for t in range(24):
        act = np.where(S_opt[:, t] > 0.5)[0]
        if len(act) == 0:
            continue
        units = []
        for a in act:
            v = S_opt[a, t]
            while v > U:
                units.append((a, U)); v -= U
            if v > 1e-9:
                units.append((a, v))
        if len(units) > len(act):
            split += 1
        nu = len(units); nv = nu * 8
        cobj = np.zeros(nv)
        for u, (gi, vol) in enumerate(units):
            for j in range(8):
                cobj[u * 8 + j] = vol * Lf[j, t]
        A = []; lbc = []; ubc = []
        for u in range(nu):
            row = np.zeros(nv); row[u * 8:(u + 1) * 8] = 1.0
            A.append(row); lbc.append(1.0); ubc.append(1.0)
        for j in range(8):
            row = np.zeros(nv)
            for u, (gi, vol) in enumerate(units):
                row[u * 8 + j] = vol
            A.append(row); lbc.append(-np.inf); ubc.append(U)
        res3 = milp(c=cobj, integrality=np.ones(nv, int), bounds=Bounds(np.zeros(nv), np.ones(nv)),
                   constraints=LinearConstraint(np.array(A), np.array(lbc), np.array(ubc)),
                   options={'disp': False, 'time_limit': 60})
        if res3.x is None or not res3.success:
            continue
        y = np.round(res3.x).astype(int).reshape(nu, 8)

```

```

        for u, (gi, vol) in enumerate(units):
            jj = int(np.argmax(y[u]))
            tplan.append({'week': t + 1, 'supplier': str(ids[idx_list[gi]]), 'transporter':
str(tids[jj]),
                        'volume': round(float(vol), 1), 'loss_rate_pct': round(float(Lf[jj, t] * 100),
3)})
            tloss += vol * Lf[jj, t]; tutil[jj] += vol
        return tplan, tloss, tutil, split

def summarize(S_opt, sel_global, tag):
    S_sel = S_opt[sel_global]
    q = S_sel / np.maximum(r_i[sel_global][:, None], 1e-9)
    recv = (S_opt * (1 - l_bar) / c_i[:, None]).sum(axis=0)
    proc = float(np.sum(S_opt * p_i[:, None]))
    trans = float(H * S_opt.sum())
    vol_by = {'A': 0.0, 'B': 0.0, 'C': 0.0}
    for k, gi in enumerate(sel_global):
        vol_by[mat[gi]] += float(S_sel[k].sum())
    tot = float(S_opt.sum())
    I = [SS]
    for t in range(24):
        I.append(I[-1] + recv[t] - W)
    return {'n': len(sel_global), 'suppliers': [str(ids[g]) for g in sel_global],
            'mat_count': {m: int((mat[sel_global] == m).sum()) for m in ['A', 'B', 'C']},
            'share': {m: vol_by[m] / tot for m in ['A', 'B', 'C']},
            'proc': round(proc, 2), 'trans': round(trans, 2), 'total': round(proc + trans, 2),
            'vol': tot, 'inventory_min': float(min(I)), 'recv_min': float(recv.min()),
            'q': q, 'S_sel': S_sel, 'S_opt': S_opt, 'sel': sel_global}

# ----- 主口径：固定问题 2 供应商，h=0.1 -----
p2_sel = np.array([np.where(ids == s)[0][0] for s in p2['selected_suppliers']])
# 构造仅含这些供应商的 LP（等效于固定 x_i）
cap_all = S_mean_active
res_fix, S_fix, sel_fix = None, None, None
# 直接复用 solve_ordering_sel 但固定选择：把其他供应商产能置 0
cap_fixed = np.zeros(len(ids)); cap_fixed[p2_sel] = S_mean_active[p2_sel]
res_fix, S_fix, sel_fix = solve_ordering_sel(len(p2_sel), cap_fixed, h=H)
fix_sum = summarize(S_fix, sel_fix, 'fix') if res_fix is not None else None

# ----- 对照口径：重新选择 22 家，h=0.1 -----
res_c, S_c, sel_c = solve_ordering_sel(22, S_mean_active, h=H)
assert res_c is not None
ctrl_sum = summarize(S_c, sel_c, 'ctrl')
tplan3, tloss3, tutil3, split3 = future_loss_assign(S_c[sel_c], sel_c)

p2A = p2['h_sensitivity'][0]['A_pct']; p2C = p2['h_sensitivity'][0]['C_pct']
p2B = 1 - p2A - p2C
ctrl = ctrl_sum
res3 = {
    'main_fixed_share': fix_sum['share'],
    'main_fixed_note': '固定 22 家供应商、含运储成本目标下，A/C 占比与问题 2 持平（容量紧张无腾挪空间）',
    'ctrl_n_suppliers': ctrl['n'],
    'ctrl_suppliers': ctrl['suppliers'],
    'ctrl_mat_count': ctrl['mat_count'],
    'vol_share': ctrl['share'],
    'procurement_cost': ctrl['proc'], 'transport_cost': ctrl['trans'], 'total_cost': ctrl['total'],
    'total_volume': ctrl['vol'],
    'overall_loss_pct': float(tloss3 / ctrl['vol'] * 100),
    'transporter_util_pct': [round(float(v / 24 / 6000 * 100), 1) for v in tutil3],
    'split_weeks': int(split3),
    'inventory_min': ctrl['inventory_min'],
    'p2_share': {'A': p2A, 'B': p2B, 'C': p2C},
    'dA_pp': round((ctrl['share']['A'] - p2A) * 100, 2),
    'dC_pp': round((ctrl['share']['C'] - p2C) * 100, 2),
    'dB_pp': round((ctrl['share']['B'] - p2B) * 100, 2),
    'vs_p2_volume_pct': round((ctrl['vol'] - p2['total_volume']) / p2['total_volume'] * 100, 3),
    'vs_p2_loss_pp': round(float(tloss3 / ctrl['vol'] * 100) - p2['overall_loss_pct'], 3),
    'h_sensitivity': p2['h_sensitivity'],
}

```

```

rows = []
for k, gi in enumerate(ctrl['sel']):
    d = {'供应商 ID': str(ids[gi]), '材料': str(mat[gi]), '履约率 r': round(float(r_i[gi]), 4),
        '周均供货能力': round(float(S_mean_active[gi]), 1)}
    for t in range(24):
        d['W%02d' % (t + 1)] = round(float(ctrl['q'][k, t]), 1)
    rows.append(d)
pd.DataFrame(rows).to_csv(os.path.join(OUT, 'problem3_order.csv'), index=False, encoding='utf-8-sig')
pd.DataFrame(tplan3).to_csv(os.path.join(OUT, 'problem3_transport.csv'), index=False, encoding='utf-8-sig')
with open(os.path.join(OUT, 'problem3_results.json'), 'w', encoding='utf-8') as f:
    json.dump(res3, f, ensure_ascii=False, indent=2)

sh = ctrl['share']
print('== 问题3 ==')
print('主口径(固定 22 家): A/B/C = %.1f%%/%.1f%%/%.1f%%' % (fix_sum['share']['A']*100,
fix_sum['share']['B']*100, fix_sum['share']['C']*100))
print('对照口径(重选 22 家): A/B/C = %.1f%%/%.1f%%/%.1f%%' % (sh['A']*100, sh['B']*100, sh['C']*100))
print('问题 2 基准(h=0): A/B/C = %.1f%%/%.1f%%/%.1f%%' % (p2A*100, p2B*100, p2C*100))
print('A 占比 %.2f pp, C 占比 %.2f pp, B 占比 %.2f pp' % (res3['dA_pp'], res3['dC_pp'], res3['dB_pp']))
print('总成本 %.2f (采购 %.2f + 运储 %.2f)' % (ctrl['total'], ctrl['proc'], ctrl['trans']))
print('总原料量 %.0f (较问题 2 占比 %.2f%%)' % (ctrl['vol'], res3['vs_p2_volume_pct']))
print('损耗率 %.3f%% (较问题 2 占比 %.3f pp)' % (res3['overall_loss_pct'], res3['vs_p2_loss_pp']))
print('saved')

```

4. step4_problem4.py

```

# -*- coding: utf-8 -*-
"""问题 4: 技改后产能提升上限 max K (现实口径 vs 理论口径) + 瓶颈识别"""
import numpy as np
import pandas as pd
import json, os
from scipy.optimize import milp, LinearConstraint, Bounds

OUT = 'output'
os.makedirs(OUT, exist_ok=True)

f1 = '附件 1 近 5 年 402 家供应商的相关数据.xlsx'
order = pd.read_excel(f1, sheet_name='企业的订货量 (m³)')
supply = pd.read_excel(f1, sheet_name='供应商的供货量 (m³)')
loss = pd.read_excel('附件 2 近 5 年 8 家转运商的相关数据.xlsx', sheet_name=0)
idc = order.columns[0]; matc = order.columns[1]; wc = order.columns[2:]
order = order.sort_values(idc).reset_index(drop=True)
supply = supply.sort_values(supply.columns[0]).reset_index(drop=True)
O = order[wc].to_numpy(float); S = supply[wc].to_numpy(float)
mat = order[matc].to_numpy(); ids = order[idc].to_numpy()
loss = loss.sort_values(loss.columns[0]).reset_index(drop=True)
L = loss[loss.columns[1:]].to_numpy(float); tids = loss[loss.columns[0]].to_numpy()

c = {'A': 0.6, 'B': 0.66, 'C': 0.72}; c_i = np.array([c[m] for m in mat])
W = 28200.0; SS = 2 * W; U_TOT = 8 * 6000.0

tot_o = O.sum(axis=1); tot_s = S.sum(axis=1)
active = (S > 0).sum(axis=1)
r_i = np.where(tot_o > 0, tot_s / tot_o, 0.0)
S_mean_active = np.where(active > 0, tot_s / np.maximum(active, 1), 0.0)
l_bar = L[L > 0].mean() / 100.0

def future_loss(j, t):
    vals = [L[j, t + 48 * y] for y in range(5) if L[j, t + 48 * y] > 0]
    if vals:
        return np.mean(vals) / 100.0
    nz = L[j][L[j] > 0]
    return (nz.mean() / 100.0) if len(nz) else 0.0
Lf = np.array([[future_loss(j, t) for t in range(24)] for j in range(8)])

```

```

with open(os.path.join(OUT, 'problem2_results.json'), encoding='utf-8') as f:
    p2 = json.load(f)

# ----- LP: max K -----
def solve_maxK(cap_raw, transport_cap=U_TOT, K_ub=60000.0):
    """max K, 变量 [K, s_00..s_{M-1,23}], 纯 LP"""
    M = len(cap_raw); T = 24; nvar = 1 + M * T
    cobj = np.zeros(nvar); cobj[0] = -1.0
    lb = np.zeros(nvar); ub = np.full(nvar, np.inf); ub[0] = K_ub
    for i in range(M):
        ub[1 + i * T:1 + (i + 1) * T] = cap_raw[i]
    rows = []; lbc = []; ubc = []
    # 库存+安全库存:  $(t+3)K - \sum_{\tau \leq t} \text{recv}_{\tau} \leq I_0$  ( $I_{t+1} = I_0 + \sum \text{recv} - (t+1)K \geq 2K$ )
    for t in range(T):
        row = np.zeros(nvar); row[0] = t + 3
        for i in range(M):
            row[1 + i * T:1 + i * T + t + 1] -= (1 - l_bar) / c_i[i]
        rows.append(row); lbc.append(-np.inf); ubc.append(SS)
    # 转运总运力:  $\sum_i s_{it} \leq \text{transport\_cap}$ 
    for t in range(T):
        row = np.zeros(nvar)
        for i in range(M):
            row[1 + i * T + t] = 1.0
        rows.append(row); lbc.append(-np.inf); ubc.append(transport_cap)
    res = milp(c=cobj, integrality=np.zeros(nvar, int), bounds=Bounds(lb, ub),
              constraints=LinearConstraint(np.array(rows), np.array(lbc), np.array(ubc)),
              options={'disp': False, 'time_limit': 300})
    if not res.success:
        return None, None
    K = res.x[0]; S_opt = res.x[1:].reshape(M, 24)
    return K, S_opt

p2_sel = np.array([np.where(ids == s)[0][0] for s in p2['selected_suppliers']])

# 现实口径: 固定 22 家供应商
cap_real = np.zeros(len(ids)); cap_real[p2_sel] = S_mean_active[p2_sel]
K_real, S_real = solve_maxK(cap_real)
# 理论口径: 全部 402 家
K_theory, S_theory = solve_maxK(S_mean_active)

# 瓶颈识别: 分别放松供应商产能/转运运力, 观察  $\Delta K$ 
def relax_test(cap_raw, relax_supplier=False, relax_transport=False):
    cap = cap_raw.copy()
    if relax_supplier:
        cap = cap * 1.1
    tc = U_TOT * 1.25 if relax_transport else U_TOT
    K, _ = solve_maxK(cap, transport_cap=tc)
    return K

K_sup10 = relax_test(S_mean_active, relax_supplier=True)
K_trans = relax_test(S_mean_active, relax_transport=True)

# 理论口径下关键约束的松紧判断
recv_theory = (S_theory * (1 - l_bar) / c_i[:, None]).sum(axis=0) if S_theory is not None else None
trans_used_theory = S_theory.sum(axis=0) if S_theory is not None else None
I_traj = [SS]
if S_theory is not None:
    for t in range(24):
        I_traj.append(I_traj[-1] + recv_theory[t] - K_theory)

# 现实口径转运方案 (损耗最少)
def future_loss_assign(S_opt, idx_list):
    U = 6000.0
    tplan = []; tloss = 0.0; tutil = np.zeros(8)
    for t in range(24):
        act = np.where(S_opt[:, t] > 0.5)[0]
        if len(act) == 0:

```

```

        continue
    units = []
    for a in act:
        v = S_opt[a, t]
        while v > U:
            units.append((a, U)); v -= U
        if v > 1e-9:
            units.append((a, v))
    nu = len(units); nv = nu * 8
    cobj = np.zeros(nv)
    for u, (gi, vol) in enumerate(units):
        for j in range(8):
            cobj[u * 8 + j] = vol * Lf[j, t]
    A = []; lbc = []; ubc = []
    for u in range(nu):
        row = np.zeros(nv); row[u * 8:(u + 1) * 8] = 1.0
        A.append(row); lbc.append(1.0); ubc.append(1.0)
    for j in range(8):
        row = np.zeros(nv)
        for u, (gi, vol) in enumerate(units):
            row[u * 8 + j] = vol
        A.append(row); lbc.append(-np.inf); ubc.append(U)
    res3 = milp(c=cobj, integrality=np.ones(nv, int), bounds=Bounds(np.zeros(nv), np.ones(nv)),
               constraints=LinearConstraint(np.array(A), np.array(lbc), np.array(ubc)),
               options={'disp': False, 'time_limit': 60})
    if res3.x is None or not res3.success:
        continue
    y = np.round(res3.x).astype(int).reshape(nu, 8)
    for u, (gi, vol) in enumerate(units):
        jj = int(np.argmax(y[u]))
        tplan.append({'week': t + 1, 'supplier': str(ids[idx_list[gi]]), 'transporter':
str(tids[jj]),
                        'volume': round(float(vol), 1), 'loss_rate_pct': round(float(Lf[jj, t] * 100),
3)})
        tloss += vol * Lf[jj, t]; tutil[jj] += vol
    return tplan, tloss, tutil

tplan4, tloss4, tutil4 = future_loss_assign(S_theory, np.arange(len(ids))) if S_theory is not None else
([], 0, np.zeros(8))

res4 = {
    'W': W,
    'K_real': float(K_real), 'K_real_increase_pct': (K_real - W) / W * 100,
    'K_theory': float(K_theory), 'K_theory_increase_pct': (K_theory - W) / W * 100,
    'supplier_total_active_product_equiv': float((S_mean_active * (1 - l_bar) / c_i).sum()),
    'supplier_total_allweek_product_equiv': float((S.mean(axis=1) * (1 - l_bar) / c_i).sum()),
    'transport_cap_raw': U_TOT,
    'transport_cap_product_equiv_minC': U_TOT / 0.72,
    'bottleneck_K_supplier_plus10': float(K_sup10),
    'bottleneck_K_transport_plus25': float(K_trans),
    'deltaK_supplier': float(K_sup10 - K_theory),
    'deltaK_transport': float(K_trans - K_theory),
    'transport_used_raw_max_weekly': float(trans_used_theory.max()) if S_theory is not None else None,
    'inventory_min': float(min(I_traj)) if S_theory is not None else None,
    'overall_loss_pct': float(tloss4 / (S_theory.sum() * 100)) if S_theory is not None and S_theory.sum()
else 0,
}
with open(os.path.join(OUT, 'problem4_results.json'), 'w', encoding='utf-8') as f:
    json.dump(res4, f, ensure_ascii=False, indent=2)

print('== 问题4 ==')
print('现实口径(固定 22 家) 最大产能 K = %.0f (%%.1f%%)' % (K_real, res4['K_real_increase_pct']))
print('理论口径(全部 402 家) 最大产能 K = %.0f (%%.1f%%)' % (K_theory, res4['K_theory_increase_pct']))
print('供应商总活跃周产能(产品当量) = %.0f, 总全周均值(产品当量) = %.0f' %
(res4['supplier_total_active_product_equiv'], res4['supplier_total_allweek_product_equiv']))
print('转运总运力 = %.0f (raw), 折 C 类产品当量上限 = %.0f' % (U_TOT,
res4['transport_cap_product_equiv_minC']))
print('瓶颈: 供应商产能+10%% -> K=%%.1f; 转运运力+25%% -> K=%%.1f' % (K_sup10, K_trans))
print('ΔK(供应商)=%.1f, ΔK(转运)=%.1f' % (res4['deltaK_supplier'], res4['deltaK_transport']))

```

```
print('理论口径周最大转运用量 %.0f (raw)' % res4['transport_used_raw_max_weekly'])
print('saved')
```

5. generate_figures.py

```
# -*- coding: utf-8 -*-
"""
```

《生产企业原材料的订购与运输》论文全部图表的统一生成脚本

=====

将原先分散在 step5/step7/step8 中的作图代码整合为单一文件，按“最终论文出现顺序”组织。运行后一次性生成 output/figures/ 下的全部图片（共 18 张，其中 13 张被论文采用、5 张为备用图）。

采用图清单（与《2021C 题论文（3）.docx》一致）：

图 1	fig_framework.png	建模总体技术路线
图 2	fig_supply_order.png	近五年周总订货量与供货量时间序列
图 3	fig_fulfill_rate.png	402 家供应商履约率分布
图 4	fig1_weights.png	评价指标客观权重对比（CRITIC vs 熵权）
图 5	fig_corr_heatmap.png	六项评价指标间相关系数热力图
图 6	fig_topsis_score.png	前 20 家供应商 TOPSIS 综合得分
图 7	fig8_concentration.png	供应商供货规模集中度（累计贡献曲线）
图 8	fig_order_heatmap.png	问题二 22 家供应商 × 24 周订货量热力图
图 9	fig3_marginal.png	供应商数量-总成本关系曲线
图 10	fig5_transporter.png	问题二各转运商周运力利用率
图 11	fig4_hsens.png	A/B/C 采购占比随运储费用 h 的变化
图 12	fig6_p2p3.png	问题二与问题三原材料采购结构对比
图 13	fig_inventory4.png	问题四理论口径下库存变化轨迹

备用图（最终论文未采用，保留以备补充说明）：

fig_radar.png	代表性供应商指标画像（S229 vs S239）
fig2_top50_mat.png	最重要的 50 家供应商材料类别分布
fig_weekly_mix.png	问题二 24 周供货量按材料构成（堆叠）
fig7_capacity.png	技术改造后最大产能（现实口径 vs 理论口径）
fig_bottleneck.png	瓶颈敏感性分析（约束松弛）

说明：图片内标题均采用“描述性标题”（不含“图 N”编号），编号由论文正文的图注负责，二者相互独立、不会冲突。

```
=====
"""
import numpy as np
import pandas as pd
import json, os
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from matplotlib import font_manager
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch, Patch
from scipy.optimize import milp, LinearConstraint, Bounds

OUT = 'output'
FIG = os.path.join(OUT, 'figures')
os.makedirs(FIG, exist_ok=True)

# ----- 中文字体 -----
for f in ['Microsoft YaHei', 'SimHei', 'SimSun']:
    try:
        font_manager.findfont(f, fallback_to_default=False)
        plt.rcParams['font.sans-serif'] = [f]
        break
    except Exception:
        continue
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['figure.dpi'] = 150

# ----- 读取结果数据 -----
```

```

with open(os.path.join(OUT, 'problem1_results.json'), encoding='utf-8') as f:
    p1 = json.load(f)
with open(os.path.join(OUT, 'problem2_results.json'), encoding='utf-8') as f:
    p2 = json.load(f)
with open(os.path.join(OUT, 'problem3_results.json'), encoding='utf-8') as f:
    p3 = json.load(f)
with open(os.path.join(OUT, 'problem4_results.json'), encoding='utf-8') as f:
    p4 = json.load(f)

feat = pd.read_csv(os.path.join(OUT, 'supplier_features.csv'))
order_df = pd.read_csv(os.path.join(OUT, 'problem2_order.csv'))

# ----- 读取原始数据 -----
f1 = '附件1 近5年402家供应商的相关数据.xlsx'
f2 = '附件2 近5年8家转运商的相关数据.xlsx'

order = pd.read_excel(f1, sheet_name='企业的订货量 (m³)')
supply = pd.read_excel(f1, sheet_name='供应商的供货量 (m³)')
idc = order.columns[0]; matc = order.columns[1]; wc = order.columns[2:]
order = order.sort_values(idc).reset_index(drop=True)
supply = supply.sort_values(supply.columns[0]).reset_index(drop=True)
O = order[wc].to_numpy(float)
S = supply[wc].to_numpy(float)
mat = order[matc].to_numpy()
ids = order[idc].to_numpy()
T = O.shape[1]

loss = pd.read_excel(f2, sheet_name=0)
loss = loss.sort_values(loss.columns[0]).reset_index(drop=True)
L = loss[loss.columns[1:]].to_numpy(float)

# ----- 常量与派生量 -----
c = {'A': 0.6, 'B': 0.66, 'C': 0.72}
c_i = np.array([c[m] for m in mat])
W = 28200.0
SS = 2 * W
U_TOT = 8 * 6000.0

tot_s = S.sum(axis=1)
active = (S > 0).sum(axis=1)
S_mean_active = np.where(active > 0, tot_s / np.maximum(active, 1), 0.0)
l_bar = L[L > 0].mean() / 100.0

def solve_maxK(cap_raw, K_ub=60000.0):
    """问题四：最大化周产能 K（线性规划），返回 (K, 供货量矩阵)"""
    M = len(cap_raw)
    Tp = 24
    nvar = 1 + M * Tp
    cobj = np.zeros(nvar); cobj[0] = -1.0
    lb = np.zeros(nvar); ub = np.full(nvar, np.inf); ub[0] = K_ub
    for i in range(M):
        ub[1 + i * Tp:1 + (i + 1) * Tp] = cap_raw[i]
    rows = []; lbc = []; ubc = []
    for t in range(Tp):
        row = np.zeros(nvar); row[0] = t + 3
        for i in range(M):
            row[1 + i * Tp:1 + i * Tp + t + 1] -= (1 - l_bar) / c_i[i]
        rows.append(row); lbc.append(-np.inf); ubc.append(SS)
    for t in range(Tp):
        row = np.zeros(nvar)
        for i in range(M):
            row[1 + i * Tp + t] = 1.0
        rows.append(row); lbc.append(-np.inf); ubc.append(U_TOT)
    res = milp(c=cobj, integrality=np.zeros(nvar, int),
              bounds=Bounds(lb, ub),
              constraints=LinearConstraint(np.array(rows), np.array(lbc), np.array(ubc)),

```



```

        options={'disp': False, 'time_limit': 300})
    if not res.success:
        return None, None
    return res.x[0], res.x[1:].reshape(M, Tp)

# =====
# 图 1 建模总体技术路线
# =====
fig, ax = plt.subplots(figsize=(7.2, 5.2))
ax.axis('off'); ax.set_xlim(0, 10); ax.set_ylim(0, 12)

def box(x, y, w, h, text, fc):
    ax.add_patch(FancyBboxPatch((x, y), w, h, boxstyle='round,pad=0.06',
                                linewidth=1.2, edgecolor='#2F5597', facecolor=fc))
    ax.text(x + w / 2, y + h / 2, text, ha='center', va='center', fontsize=9.5, color='#1F3864')

def arrow(x1, y1, x2, y2):
    ax.add_patch(FancyArrowPatch((x1, y1), (x2, y2), arrowstyle='->',
                                mutation_scale=14, linewidth=1.3, color='#404040'))

box(3.4, 10.8, 3.2, 1.0, '数据清洗与特征工程\n(仅 0 为哨兵值)', '#DDEBF7')
box(3.4, 8.9, 3.2, 1.0, '问题一: 供应商重要性评价\nCRITIC-TOPSIS', '#DDEBF7')
box(0.4, 8.9, 2.6, 1.0, '折算系数\nc_A/c_B/c_C', '#FCE4D6')
box(7.0, 8.9, 2.6, 1.0, '供货率/损耗率\n预测子模型', '#FCE4D6')
box(3.4, 6.9, 3.2, 1.0, '问题二: 最少供应商数\n+ 订购 + 转运 (MILP)', '#E2EFDA')
box(0.4, 6.9, 2.6, 1.0, '可持续供货能力\n活跃周均值', '#FCE4D6')
box(7.0, 6.9, 2.6, 1.0, '50 家候选池\n(问题一结果)', '#FCE4D6')
box(3.4, 4.9, 3.2, 1.0, '问题三: 压缩成本\n多 A 少 C (多目标)', '#E2EFDA')
box(3.4, 2.9, 3.2, 1.0, '问题四: 产能提升上限\nmax K + 瓶颈识别', '#E2EFDA')
box(3.4, 0.9, 3.2, 1.0, '结果分析与方案输出\n(逐周订购/转运方案)', '#FFF2CC')

arrow(5.0, 10.8, 5.0, 9.9)
arrow(2.0, 8.9, 3.4, 9.4); arrow(8.0, 8.9, 6.6, 9.4)
arrow(5.0, 8.9, 5.0, 7.9)
arrow(2.0, 6.9, 3.4, 7.4); arrow(8.0, 6.9, 6.6, 7.4)
arrow(5.0, 6.9, 5.0, 5.9)
arrow(5.0, 4.9, 5.0, 3.9)
arrow(5.0, 2.9, 5.0, 1.9)
ax.set_title('建模总体技术路线', fontsize=12, pad=8)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_framework.png')); plt.close(fig)

# =====
# 图 2 近五年周总订货量与供货量时间序列
# =====
otot = 0.sum(axis=0); stot = S.sum(axis=0)
fig, ax = plt.subplots(figsize=(7, 3.6))
wks = np.arange(1, T + 1)
ax.plot(wks, otot, color='#4472C4', linewidth=1.0, label='周总订货量')
ax.plot(wks, stot, color='#ED7D31', linewidth=1.0, label='周总供货量')
for y in range(48, T + 1, 48):
    ax.axvline(y, color='gray', linestyle=':', linewidth=0.7)
ax.set_xlabel('周次 (共 240 周 / 5 年)'); ax.set_ylabel('周订货量 / 供货量 (m³)')
ax.set_title('近五年周总订货量与供货量时间序列')
ax.legend(loc='upper right', fontsize=9); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_supply_order.png')); plt.close(fig)

# =====
# 图 3 402 家供应商履约率分布
# =====
r_i = feat['履约率'].values
fig, ax = plt.subplots(figsize=(7, 3.6))

```

```

ax.hist(r_i, bins=40, color='#4472C4', edgecolor='white', alpha=0.9)
ax.axvline(r_i.mean(), color='#C00000', linestyle='--', linewidth=1.2,
           label='均值 %.3f' % r_i.mean())
ax.set_xlabel('供应商履约率（总供货量 / 总订货量）'); ax.set_ylabel('供应商数量')
ax.set_title('402 家供应商履约率分布')
ax.legend(); ax.grid(alpha=0.3, axis='y')
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_fulfill_rate.png')); plt.close(fig)

# =====
# 图 4 评价指标客观权重对比（CRITIC vs 熵权）
# =====
ind = p1['ind_names']
w_c = p1['critic_w']; w_e = p1['entropy_w']
x = np.arange(len(ind))
fig, ax = plt.subplots(figsize=(7, 4))
w = 0.38
ax.bar(x - w / 2, w_c, w, label='CRITIC 权重', color='#4472C4')
ax.bar(x + w / 2, w_e, w, label='熵权', color='#ED7D31')
ax.set_xticks(x); ax.set_xticklabels(ind)
ax.set_ylabel('权重'); ax.set_title('评价指标客观权重对比（CRITIC vs 熵权）')
ax.legend()
for i, v in enumerate(w_c):
    ax.text(x[i] - w / 2, v + 0.01, '%.3f' % v, ha='center', fontsize=7)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig1_weights.png')); plt.close(fig)

# =====
# 图 5 六项评价指标间相关系数热力图
# =====
cols = ['X1 规模', 'X2 波动', 'X3 可靠', 'X4 持续', 'X5 应急', 'X6 断供']
X6 = feat[cols].to_numpy(float)
corr = np.corrcoef(X6, rowvar=False)
fig, ax = plt.subplots(figsize=(6, 4.8))
im = ax.imshow(corr, cmap='RdBu_r', vmin=-1, vmax=1)
ax.set_xticks(range(6)); ax.set_xticklabels(cols, fontsize=8)
ax.set_yticks(range(6)); ax.set_yticklabels(cols, fontsize=8)
for i in range(6):
    for j in range(6):
        ax.text(j, i, '%.2f' % corr[i, j], ha='center', va='center',
                fontsize=8, color='white' if abs(corr[i, j]) > 0.6 else 'black')
ax.set_title('六项评价指标间相关系数热力图', fontsize=11)
fig.colorbar(im, shrink=0.8)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_corr_heatmap.png')); plt.close(fig)

# =====
# 图 6 前 20 家供应商 TOPSIS 综合得分（按材料着色）
# =====
ids20 = p1['top50_ids'][:20]
mat20 = p1['top50_materials'][:20]
sc20 = p1['top50_scores'][:20]
colors = {'A': '#4472C4', 'B': '#ED7D31', 'C': '#70AD47'}
fig, ax = plt.subplots(figsize=(7, 4.4))
ypos = np.arange(len(ids20))[:-1]
bars = ax.barh(ypos, sc20, color=[colors[m] for m in mat20], height=0.62)
ax.set_yticks(ypos); ax.set_yticklabels(ids20, fontsize=8)
ax.set_xlabel('TOPSIS 综合得分 S'); ax.set_title('前 20 家供应商 TOPSIS 综合得分（按材料着色）')
for b, s in zip(bars, sc20):
    ax.text(s + 0.005, b.get_y() + b.get_height() / 2, '%.3f' % s, va='center', fontsize=7)
ax.legend(handles=[Patch(color=colors[m], label=m + ' 类') for m in ['A', 'B', 'C']], fontsize=9)
ax.grid(alpha=0.3, axis='x')
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_topsis_score.png')); plt.close(fig)

# =====
# 图 7 供应商供货规模集中度（累计贡献曲线）
# =====
tot = feat['总供货量 m3'].sort_values(ascending=False).values
cum = np.cumsum(tot) / tot.sum()
fig, ax = plt.subplots(figsize=(6.5, 4))

```

```

ax.plot(range(1, len(tot) + 1), cum * 100, color='#4472C4', linewidth=2)
ax.axhline(50, color='gray', linestyle='--', linewidth=0.8)
ax.axhline(80, color='gray', linestyle='--', linewidth=0.8)
for k, lab in [(22, '前 22 家'), (50, '前 50 家')]:
    ax.plot([k], [cum[k - 1] * 100], 'o', color='#C00000')
    ax.annotate('%s=%0f%%' % (lab, cum[k - 1] * 100), xy=(k, cum[k - 1] * 100),
                xytext=(k + 20, cum[k - 1] * 100 - 8),
                arrowprops=dict(arrowstyle='->', color='#C00000'), color='#C00000', fontsize=9)
ax.set_xlabel('供应商数 (按总供货量降序)'); ax.set_ylabel('累计供货量占比 (%)')
ax.set_title('供应商供货规模集中度 (累计贡献曲线)')
ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig8_concentration.png')); plt.close(fig)

# =====
# 图 8 问题二 22 家供应商 × 24 周订货到热力图
# =====
q = order_df[['W{i:02d}' for i in range(1, 25)]].to_numpy(float)
mats = order_df['材料'].to_numpy()
labels = order_df['供应商 ID'].to_numpy()
fig, ax = plt.subplots(figsize=(7, 4.8))
im = ax.imshow(q, cmap='Blues', aspect='auto')
ax.set_xticks(range(0, 24, 2)); ax.set_xticklabels(['W{i}' for i in range(1, 25, 2)], fontsize=7)
ax.set_yticks(range(len(labels))); ax.set_yticklabels(labels, fontsize=7)
ax.set_xlabel('周次'); ax.set_ylabel('供应商')
ax.set_title('问题二 22 家供应商 × 24 周订货到热力图 (m³)')
fig.colorbar(im, shrink=0.8)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_order_heatmap.png')); plt.close(fig)

# =====
# 图 9 供应商数量-总成本关系曲线
# =====
mc = p2['marginal_cost']
Ns = [m['N'] for m in mc]; costs = [m['cost'] for m in mc]
fig, ax = plt.subplots(figsize=(6.5, 4))
ax.plot(Ns, costs, 'o-', color='#4472C4', linewidth=2, markersize=5)
ax.set_xlabel('供应商数量 N'); ax.set_ylabel('总成本 (相对单位)')
ax.set_title('供应商数量-总成本关系 (N=22 为最少供应商数)')
ax.axvline(22, color='red', linestyle='--', linewidth=1, alpha=0.7)
ax.annotate('N=22 (最少)', xy=(22, costs[0]), xytext=(24.5, costs[0] + 40),
            arrowprops=dict(arrowstyle='->', color='red'), color='red', fontsize=9)
ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig3_marginal.png')); plt.close(fig)

# =====
# 图 10 问题二各转运商周运力利用率
# =====
tnames = ['T1', 'T2', 'T3', 'T4', 'T5', 'T6', 'T7', 'T8']
util = p2['transporter_util_pct']
fig, ax = plt.subplots(figsize=(6.5, 4))
bars = ax.bar(tnames, util, color=['#4472C4' if u < 99 else '#C00000' for u in util])
ax.axhline(100, color='gray', linestyle='--', linewidth=0.8)
ax.set_xlabel('转运商'); ax.set_ylabel('周运力利用率 (%)')
ax.set_title('问题二各转运商周运力利用率 (T3 接近满载)')
for b, u in zip(bars, util):
    ax.text(b.get_x() + b.get_width() / 2, u + 1.5, '%.1f%%' % u, ha='center', fontsize=8)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig5_transporter.png')); plt.close(fig)

# =====
# 图 11 A/B/C 采购占比随仓储费用 h 的变化
# =====
hs = p2['h_sensitivity']
hvals = [s['h'] for s in hs]
As = [s['A_pct'] * 100 for s in hs]
Cs = [s['C_pct'] * 100 for s in hs]
Bs = [s['B_pct'] * 100 for s in hs]
fig, ax = plt.subplots(figsize=(6.5, 4))
ax.plot(hvals, As, 'o-', label='A 类占比', color='#4472C4', linewidth=2)

```

```

ax.plot(hvals, Bs, 's-', label='B 类占比', color='#ED7D31', linewidth=2)
ax.plot(hvals, Cs, '^-', label='C 类占比', color='#70AD47', linewidth=2)
ax.set_xlabel('单位运储费用 h (相对 C 类采购价)'); ax.set_ylabel('采购量占比 (%)')
ax.set_title('A/B/C 采购占比随运储费用 h 的变化 (内生"多 A 少 C"机制)')
ax.legend(); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig4_hsens.png')); plt.close(fig)

# =====
# 图 12 问题二与问题三原材料采购结构对比
# =====
p2s = p3['p2_share']; p3s = p3['vol_share']
x = np.arange(3); w = 0.38
fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(x - w / 2, [p2s['A'] * 100, p2s['B'] * 100, p2s['C'] * 100], w,
      label='问题 2 (仅采购成本)', color='#A5A5A5')
ax.bar(x + w / 2, [p3s['A'] * 100, p3s['B'] * 100, p3s['C'] * 100], w,
      label='问题 3 (含运储成本)', color='#4472C4')
ax.set_xticks(x); ax.set_xticklabels(['A 类', 'B 类', 'C 类'])
ax.set_ylabel('采购量占比 (%)'); ax.set_title('问题 2 与问题 3 原材料采购结构对比 (A 增 C 减)')
ax.legend()
for i in range(3):
    ax.text(x[i] - w / 2, [p2s['A'] * 100, p2s['B'] * 100, p2s['C'] * 100][i] + 1,
           '%.1f' % [p2s['A'] * 100, p2s['B'] * 100, p2s['C'] * 100][i], ha='center', fontsize=8)
    ax.text(x[i] + w / 2, [p3s['A'] * 100, p3s['B'] * 100, p3s['C'] * 100][i] + 1,
           '%.1f' % [p3s['A'] * 100, p3s['B'] * 100, p3s['C'] * 100][i], ha='center', fontsize=8)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig6_p2p3.png')); plt.close(fig)

# =====
# 图 13 问题四理论口径下库存变化轨迹
# =====
K, S_theory = solve_maxK(S_mean_active)
# 最小可行库存轨迹: 首周将安全库存由 2W 一次性提升至 2K, 此后每周恰好补充 K 维持
recv1 = 3 * K - 2 * W
I = [SS, 2 * K] + [2 * K] * 23

fig, ax = plt.subplots(figsize=(6.6, 3.8))
wk = np.arange(0, 25)
ax.plot(wk, I, 'o-', color='#4472C4', linewidth=2, markersize=4, label='最小可行库存 I_t')
ax.axhline(2 * K, color='#C00000', linestyle='--', linewidth=1.2,
          label='安全库存 2K = %.0f' % (2 * K))
ax.axhline(SS, color='gray', linestyle=':', linewidth=1.2, label='期初 2W = %.0f' % SS)
ax.annotate('首周堆积\n%.0f → %.0f' % (SS, 2 * K), xy=(1, 2 * K),
          xytext=(3, 2 * K + 400),
          arrowprops=dict(arrowstyle='->', color='#C00000'), color='#C00000', fontsize=9)
ax.set_xlabel('周次 t'); ax.set_ylabel('周末库存 (等效产品当量 m³)')
ax.set_title('问题四理论口径下库存变化轨迹 (首周完成安全库存堆积)', fontsize=11)
ax.legend(fontsize=8); ax.grid(alpha=0.3)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_inventory4.png')); plt.close(fig)

# =====
# 以下为备用图 (最终论文未采用, 保留以备需要时补充)
# =====

# ----- 备用: 代表性供应商指标画像 (S229 vs S239 雷达图) -----
cols = ['X1 规模', 'X2 波动', 'X3 可靠', 'X4 持续', 'X5 应急', 'X6 断供']
pos_idx = [0, 2, 3, 4]; neg_idx = [1, 5]
raw = feat[cols].to_numpy(float)
norm = np.zeros_like(raw)
for k in range(6):
    mn, mx = raw[:, k].min(), raw[:, k].max()
    if k in pos_idx:
        norm[:, k] = (raw[:, k] - mn) / (mx - mn)
    else:
        norm[:, k] = (mx - raw[:, k]) / (mx - mn)
id_col = feat.columns[0]

```

```

fmap = {feat[id_col].iloc[i]: i for i in range(len(feat))}
labels = ['规模', '波动(反向)', '可靠', '持续', '应急', '断供(反向)']
angles = np.linspace(0, 2 * np.pi, 6, endpoint=False).tolist()
angles += angles[:1]

fig, ax = plt.subplots(figsize=(5.6, 5.6), subplot_kw=dict(polar=True))
for sid, cc, lab in [('S229', '#4472C4', '第1名 S229'), ('S239', '#ED7D31', '第50名 S239')]:
    vals = norm[fmap[sid]].tolist(); vals += vals[:1]
    ax.plot(angles, vals, 'o-', linewidth=2, color=cc, label=lab)
    ax.fill(angles, vals, alpha=0.1, color=cc)
ax.set_xticks(angles[:-1]); ax.set_xticklabels(labels, fontsize=9)
ax.set_ylim(0, 1.05)
ax.set_title('代表性供应商指标画像 (S229 vs S239)', fontsize=12, pad=20)
ax.legend(loc='upper right', bbox_to_anchor=(1.25, 1.12), fontsize=9)
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_radar.png')); plt.close(fig)

# ----- 备用: 最重要的 50 家供应商材料类别分布 -----
pie_mats = ['A', 'B', 'C']
vals = [p1['top50_mat_dist'][m] for m in pie_mats]
fig, ax = plt.subplots(figsize=(5, 4))
ax.pie(vals, labels=['A类(%d家)' % vals[0], 'B类(%d家)' % vals[1], 'C类(%d家)' % vals[2]],
        autopct='%1.1f%%', colors=['#4472C4', '#ED7D31', '#A5A5A5'], startangle=90)
ax.set_title('最重要的 50 家供应商材料类别分布')
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig2_top50_mat.png')); plt.close(fig)

# ----- 备用: 问题二 24 周供货量按材料构成(堆叠) -----
r_map = dict(zip(order_df['供应商ID'], order_df['履约率r']))
sup_week = np.zeros((len(order_df), 24))
for k, (_, row) in enumerate(order_df.iterrows()):
    r = row['履约率r']
    sup_week[k] = row[['W{i:02d}' for i in range(1, 25)]] * r
mat_week = {'A': sup_week[mats == 'A'].sum(axis=0),
            'B': sup_week[mats == 'B'].sum(axis=0),
            'C': sup_week[mats == 'C'].sum(axis=0)}
wks24 = np.arange(1, 25)
fig, ax = plt.subplots(figsize=(7, 3.8))
ax.bar(wks24, mat_week['A'], color='#4472C4', label='A 类')
ax.bar(wks24, mat_week['B'], bottom=mat_week['A'], color='#ED7D31', label='B 类')
ax.bar(wks24, mat_week['C'], bottom=mat_week['A'] + mat_week['B'], color='#70AD47', label='C 类')
ax.axhline(28200, color='red', linestyle='--', linewidth=1, label='周产能 W=28200')
ax.set_xlabel('周次'); ax.set_ylabel('周供货量 (m³)')
ax.set_title('问题二 24 周供货量按材料构成(堆叠)')
ax.legend(fontsize=8, ncol=4); ax.grid(alpha=0.3, axis='y')
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_weekly_mix.png')); plt.close(fig)

# ----- 备用: 技术改造后最大产能(现实口径 vs 理论口径) -----
fig, ax = plt.subplots(figsize=(6, 4))
labels = ['当前产能\nW=28200', '现实口径\n(固定 22 家)', '理论口径\n(全部 402 家)']
vals = [28200, p4['K_real'], p4['K_theory']]
bars = ax.bar(labels, vals, color=['#A5A5A5', '#ED7D31', '#4472C4'], width=0.55)
ax.axhline(28200, color='gray', linestyle='--', linewidth=0.8)
for b, v in zip(bars, vals):
    ax.text(b.get_x() + b.get_width() / 2, v + 300,
            '%.0f\n(+%.1f%%)' % (v, (v - 28200) / 28200 * 100), ha='center', fontsize=9)
ax.set_ylabel('最大周产能 K (m³)'); ax.set_title('技术改造后最大产能(现实口径 vs 理论口径)')
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig7_capacity.png')); plt.close(fig)

# ----- 备用: 瓶颈敏感性分析(约束松弛) -----
fig, ax = plt.subplots(figsize=(6, 4))
labels = ['基线\n(理论口径)', '供应商能力\n+10%', '转运运力\n+25%']
vals = [p4['K_theory'], p4['bottleneck_K_supplier_plus10'], p4['bottleneck_K_transport_plus25']]
bars = ax.bar(labels, vals, color=['#A5A5A5', '#4472C4', '#ED7D31'], width=0.5)
for b, v in zip(bars, vals):
    ax.text(b.get_x() + b.get_width() / 2, v + 150, '%.0f' % v, ha='center', fontsize=10)
ax.axhline(p4['K_theory'], color='gray', linestyle='--', linewidth=0.8)
ax.set_ylabel('最大产能 K (m³)'); ax.set_title('瓶颈敏感性分析(约束松弛)')
fig.tight_layout(); fig.savefig(os.path.join(FIG, 'fig_bottleneck.png')); plt.close(fig)

```

```
# ----- 汇总输出 -----  
print('figures saved to', FIG)  
print('total files:', len([f for f in os.listdir(FIG) if f.endswith('.png')]))  
for fn in sorted(os.listdir(FIG)):  
    if fn.endswith('.png'):  
        print(' ', fn)
```

附录 C：支撑材料文件说明

本论文支撑材料压缩包包含以下文件：

1. step1_problem1.py：问题一供应商综合评价 CRITIC 赋权求解代码
2. step2_problem2.py：问题二供应商选择+订货转运混合整数规划代码
3. step3_problem3.py：问题三多 A 少 C 成本优化与字典序损耗最小代码
4. step4_problem4.py：问题四产能上限最大化与瓶颈分析代码
5. generate_figures.py：论文图表绘制代码
6. step6_paper.py、step8_extra.py、step9_revise.py：辅助计算与敏感性分析代码
7. 附件 1 近 5 年 402 家供应商的相关数据.xlsx：原始供应商数据
8. 附件 2 近 5 年 8 家转运商的相关数据.xlsx：原始转运商数据
9. 附件 A 订购方案数据结果.xlsx：各问题订货求解结果明细
10. 附件 B 转运方案数据结果.xlsx：转运指派方案详细结果